



RAMRAO ADIK INSTITUTE OF TECHNOLOGY

Nerul, Navi Mumbai – 400 706
D. Y. Patil Deemed to be University

DEPARTMENT OF COMPUTER ENGINEERING

ACADEMIC LABORATORY DOCUMENTATION

SBL – OOP JAVA

LABORATORY RECORD

CANDIDATE CREDENTIALS & EXAMINATION RECORD

Candidate Name	Anish Vyapari
Roll Number	25CA1012
PRN Identifier	DY25ENGU0AIM012
Class & Division	SE (Computer Engineering) – Division A / A1
Semester & Term	Semester 3
Academic Session	Academic Year 2024 – 2025

LIST OF EXPERIMENTS / LABORATORY INDEX

Course Code: SBL – OOP Java | Department of Computer Engineering | Semester 3

Sr. No.	Experiment Title	Page No.
1	Demonstrate control structure: if-else, switch, for, while loop and construct class and object.	3
2	Implement method overloading and constructor overloading.	9
3	Build a program on 1D and 2D arrays.	14
4	Implementation of string manipulation functions and write a program to check whether it is palindrome or not.	19
5	Implement a single and multilevel inheritance (use super keyword).	24
6	Build an interface demonstrating concept of multiple inheritance.	29
7	Write a program to demonstrate exception handling using try, catch, throw, throws and finally.	34
8	Design and implement user defined exception handling.	38
9	Construct the component and container class to construct/design GUI.	43
10	Build a dynamic feature using Java based script.	51
11	Capstone Project: Smart Banking Transaction and Account Portfolio Management System (SB-TMS)	58
—	OUTPUT PRINT SECTION (Consolidated Outputs for Experiments 1 to 11)	67

EXPERIMENT 1

Demonstrate control structure: if-else, switch, for, while loop and construct class and object.

■ AIM

To write and execute a Java program to demonstrate decision-making and looping control structures (if-else, switch, for loop, while loop) and construct a class with member variables, methods, and instantiate objects.

■ OBJECTIVE

- Understand the Object-Oriented paradigm and class-object architectural relationships in the Java Virtual Machine (JVM).
- Implement two-way and multi-way branching constructs (if-else ladder and switch statement) to evaluate logical conditions.
- Implement entry-controlled iterative constructs (for and while loops) for deterministic traversal and condition-driven execution.
- Construct a fully encapsulated Java class with constructors, instance methods, and instantiate active objects on the Java heap.

■ SOFTWARE REQUIRED

Java Development Kit (JDK 21 LTS), Visual Studio Code / Eclipse IDE, Windows PowerShell / Command Prompt.

■ THEORY

1. Fundamentals of Object-Oriented Programming (OOP) in Java

Java is an object-oriented, class-based, concurrent, and strongly typed programming language designed around the foundational philosophy of *"Write Once, Run Anywhere"* (WORA). In Java, computation is structured around **objects** rather than actions, and data rather than logic. An object represents a discrete entity encapsulating state (represented by field variables) and behavior (represented by member methods). The **class** serves as a user-defined blueprint, prototype, or template from which individual objects are instantiated at runtime.

When a Java source file (`.java`) is compiled using `javac`, it is transformed into architecture-neutral bytecode (`.class` files). At runtime, the Java Virtual Machine (JVM) loads bytecode into memory through its ClassLoader subsystem. Memory is partitioned into functional regions: the **Method Area** (storing class metadata and bytecode), the **Java Heap** (dynamic memory where all class objects and instance variables reside), and the **Java Stack** (allocating individual stack frames for thread execution, local variables, and object references).

2. Class Definition, Object Instantiation, and Memory Lifecycle

A class in Java is declared using the `class` keyword followed by an identifier. It encapsulates member attributes (which determine the state) and methods (which define operations). An object is physically created using the `new` keyword, which dynamically allocates required memory on the heap and invokes a constructor to initialize the instance state:

```
// Class declaration syntax
class Student {
    private int rollNumber;           // Instance variable (Heap allocated)
    private String name;              // Reference variable pointing to String in Heap

    public Student(int r, String n) { // Parameterized constructor
        this.rollNumber = r;
        this.name = n;
    }
}

// Object instantiation syntax
Student s1 = new Student(1012, "Anish Vyapari");
```

In the statement above, `s1` is a reference variable allocated on the call stack containing the 32-bit or 64-bit memory address of the concrete `Student` object residing in heap space. When an object is no longer reachable by any active reference in the application, it becomes eligible for automatic garbage collection (GC), preventing memory leaks without manual memory management.

3. Decision-Making Control Structures: if-else and switch

Decision-making structures direct program execution along alternative paths based on conditional evaluations:

- **if-else Ladder:** Evaluates boolean expressions sequentially. The `if` block executes only if its condition evaluates to `true`. Subsequent `else if` clauses provide alternate testing conditions, terminating with an optional `else` fallback. Java supports short-circuit logical operators (`&&`, `||`), where evaluation aborts as soon as the outcome is deterministically known, optimizing runtime execution.
- **switch Statement:** A multi-branch selection construct that compares an expression against discrete scalar constant values (`case` labels). In Java, switch expressions support primitive types (`byte`, `short`, `char`, `int`), their wrapper classes, `String` literals, and `enum` types. The JVM optimizes switch statements into bytecode using either `tableswitch` (for dense constant ranges with O(1) indexed jump table lookup) or `lookupswitch` (for sparse constants using binary search). The `break` keyword prevents unintended fall-through to succeeding cases, while `default` catches unmatched expressions.

4. Iterative Control Structures: for and while Loops

Repetitive execution of code blocks is governed by iterative loop constructs:

- **for Loop:** An entry-controlled loop optimized for deterministic iteration where the iteration count is known a priori. It integrates three expressions: initialization, boolean termination condition, and update expression (e.g., `for(int i = 0; i < n; i++)`). The condition is evaluated prior to every iteration; if false, the loop terminates immediately.
- **while Loop:** An entry-controlled loop suited for indefinite iteration where execution continues as long as a governing condition remains true. The condition is tested at the top of each iteration, meaning the loop body may execute zero times if the condition is initially false. It is widely employed in stream processing, digit manipulation, and state-machine polling loops.

Together, classes, objects, branching logic, and iterative constructs form the bedrock of robust, maintainable Java software systems.

■ ALGORITHM

1. Start the program.

2. Define class Student with private attributes: rollNumber (int), name (String), subjectMarks (int array), and attendancePercentage (double).
3. Define a parameterized constructor Student(rollNumber, name, subjectMarks, attendancePercentage) to initialize object instance variables.
4. Implement method checkEligibility(): evaluate attendancePercentage using an if-else-if ladder. If attendance $\geq$ 75%, mark eligible; if 65% to 74.9%, mark conditionally eligible with condonation; otherwise, mark defaulter / ineligible.
5. Implement method calculateAverageMarks(): initialize totalMarks = 0 and highestMark = subjectMarks[0]. Use a for loop from index 0 to array length - 1 to traverse subjectMarks, accumulate total, update highest mark, compute average, and return average.
6. Implement method assignGradeAndRemarks(average): compute gradeBand = (int) average / 10. Pass gradeBand into a switch construct to assign letter grades ('O', 'A', 'B', 'C', 'D', 'F') and display performance remarks.
7. Implement method displayStudentAudit(): initialize tempRoll with rollNumber and digitSum = 0. Use a while loop with condition tempRoll > 0 to compute digit checksum via remainder and integer division.
8. In main(), instantiate Student objects (e.g., Anish Vyapari with roll 1012, Rohan Sharma with roll 1045) using the new operator.
9. Invoke displayProfile(), checkEligibility(), calculateAverageMarks(), assignGradeAndRemarks(), and displayStudentAudit() on each instantiated object.
10. Display formatted execution results to the console.
11. Stop.

■ PROGRAM

```
// Experiment 1: Demonstrate control structure: if-else, switch, for, while loop and construct
class and object.
// Subject: SBL - OOP Java
// Student: Anish Vyapari | Roll No: 25CA1012 | PRN: DY25ENGU0AIM012

class Student {
    // Member variables (attributes)
    private int rollNumber;
    private String name;
    private int[] subjectMarks;
    private double attendancePercentage;

    // Constructor to initialize object state
    public Student(int rollNumber, String name, int[] subjectMarks, double attendancePercentage) {
        this.rollNumber = rollNumber;
        this.name = name;
        this.subjectMarks = subjectMarks;
        this.attendancePercentage = attendancePercentage;
    }

    // Method to evaluate exam eligibility using if-else control structure
    public boolean checkEligibility() {
        System.out.println("\n--- 1. Evaluating Eligibility (if-else structure) ---");
        if (attendancePercentage  $\geq$  75.0) {
            System.out.println("Status: Eligible for Semester Examination (Attendance: " +
attendancePercentage + "%  $\geq$  75%)");
            return true;
        } else if (attendancePercentage  $\geq$  65.0) {
            System.out.println("Status: Conditionally Eligible with Medical Condonation
(Attendance: " + attendancePercentage + "%)");
            return true;
        }
    }
}
```

```

    } else {
        System.out.println("Status: Defaulter / Not Eligible for Examination (Attendance: " +
attendancePercentage + "% < 65%)");
        return false;
    }
}

// Method to calculate total and average marks using for loop
public double calculateAverageMarks() {
    System.out.println("\n--- 2. Calculating Academic Performance (for loop) ---");
    int totalMarks = 0;
    int highestMark = subjectMarks[0];

    // 'for' loop iterating through the marks array
    for (int i = 0; i < subjectMarks.length; i++) {
        System.out.println(" Subject " + (i + 1) + " Marks: " + subjectMarks[i]);
        totalMarks += subjectMarks[i];
        if (subjectMarks[i] > highestMark) {
            highestMark = subjectMarks[i];
        }
    }

    double average = (double) totalMarks / subjectMarks.length;
    System.out.println("Total Marks Obtained: " + totalMarks + " / " + (subjectMarks.length *
100));
    System.out.printf("Calculated Average: %.2f%%\n", average);
    System.out.println("Highest Subject Mark: " + highestMark);
    return average;
}

// Method to assign letter grade and provide remarks using switch structure
public void assignGradeAndRemarks(double average) {
    System.out.println("\n--- 3. Assigning Grade and Feedback (switch statement) ---");
    char gradeLetter;
    int gradeBand = (int) average / 10;

    switch (gradeBand) {
        case 10:
        case 9:
            gradeLetter = 'O'; // Outstanding
            break;
        case 8:
            gradeLetter = 'A'; // Excellent
            break;
        case 7:
            gradeLetter = 'B'; // Very Good
            break;
        case 6:
            gradeLetter = 'C'; // Good
            break;
        case 5:
        case 4:
            gradeLetter = 'D'; // Pass
            break;
        default:
            gradeLetter = 'F'; // Fail
            break;
    }

    System.out.println("Letter Grade Assigned: " + gradeLetter);

    // Enhanced switch feedback
    switch (gradeLetter) {
        case 'O':

```

```

        System.out.println("Faculty Remark: Outstanding Performance! Recommended for Merit Honors.");
        break;
        case 'A':
            System.out.println("Faculty Remark: Excellent achievement! Demonstrates strong conceptual mastery.");
            break;
        case 'B':
            System.out.println("Faculty Remark: Very Good effort. Consistent lab and theory performance.");
            break;
        case 'C':
            System.out.println("Faculty Remark: Satisfactory performance. Potential for further improvement.");
            break;
        case 'D':
            System.out.println("Faculty Remark: Passed. Remedial sessions recommended in core modules.");
            break;
        case 'F':
            System.out.println("Faculty Remark: Failed. Must reappear for supplementary assessment.");
            break;
        default:
            System.out.println("Faculty Remark: Invalid grade evaluation.");
            break;
    }
}

// Method to demonstrate while loop for digit summation and validation
public void displayStudentAudit() {
    System.out.println("\n--- 4. Integrity Check / Digit Verification (while loop) ---");
    int tempRoll = this.rollNumber;
    int digitSum = 0;

    // 'while' loop runs until roll number digits are completely summed
    while (tempRoll > 0) {
        int remainder = tempRoll % 10;
        digitSum += remainder;
        tempRoll = tempRoll / 10;
    }
    System.out.println("Student Roll No: " + this.rollNumber);
    System.out.println("Calculated Checksum of Roll Number: " + digitSum);
    System.out.println("Record Validation Status: Verified & Recorded in System.");
}

// Display summary
public void displayProfile() {
    System.out.println("=====");
    System.out.println("          STUDENT ACADEMIC PROFILE RECORD          ");
    System.out.println("=====");
    System.out.println("Roll Number : " + rollNumber);
    System.out.println("Name       : " + name);
    System.out.println("Attendance  : " + attendancePercentage + "%");
}

}

public class Exp1ControlStructures {
    public static void main(String[] args) {
        System.out.println("=== RAIT Nerul - SBL OOP Java Lab: Experiment 1 ===");
        System.out.println("Candidate: Anish Vyapari | Roll No: 25CA1012");
        System.out.println("Topic: Control Structures and Class/Object Construction\n");

        // Constructing object 1: Anish Vyapari
    }
}

```

```

        int[] marksAnish = {92, 88, 95, 89, 94};
        Student student1 = new Student(1012, "Anish Vyapari", marksAnish, 91.5);
        student1.displayProfile();
        boolean isEligible = student1.checkEligibility();
        if (isEligible) {
            double avg = student1.calculateAverageMarks();
            student1.assignGradeAndRemarks(avg);
        }
        student1.displayStudentAudit();

        System.out.println("\n-----");

        // Constructing object 2: Demonstrating conditional branch
        int[] marksStudent2 = {54, 48, 62, 58, 50};
        Student student2 = new Student(1045, "Rohan Sharma", marksStudent2, 68.0);
        student2.displayProfile();
        student2.checkEligibility();
        double avg2 = student2.calculateAverageMarks();
        student2.assignGradeAndRemarks(avg2);
        student2.displayStudentAudit();
        System.out.println("=====");
    }
}

```

■ CONCLUSION

Successfully constructed an object-oriented Java class and instantiated concrete objects on the heap. Verified the functional behavior and operational control flow of if-else conditional branches, multi-way switch statements, entry-controlled for loops, and condition-driven while loops in an academic student evaluation system.

EXPERIMENT 2

Implement method overloading and constructor overloading.

■ AIM

To implement compile-time polymorphism in Java using method overloading (defining methods with identical names but differing parameter signatures) and constructor overloading.

■ OBJECTIVE

- Understand the concept of compile-time (static) polymorphism and static method binding in the Java compiler.
- Implement constructor overloading to support multiple object initialization modalities for geometric solids.
- Implement method overloading by varying the number, data types, and sequence of method parameters.
- Analyze type promotion rules and method signature resolution mechanics in the Java Virtual Machine.

■ SOFTWARE REQUIRED

Java Development Kit (JDK 21 LTS), Visual Studio Code / Eclipse IDE, Windows Terminal.

■ THEORY

1. Concept of Polymorphism in Object-Oriented Programming

Polymorphism, derived from the Greek words *poly* (many) and *morph* (forms), is the capability in object-oriented programming that allows an entity—such as a method, operator, or constructor—to behave differently depending on the context in which it is invoked. Java facilitates two fundamental categories of polymorphism:

1. **Compile-Time Polymorphism (Static / Early Binding):** Resolved during source compilation. Exemplified by *method overloading* and *constructor overloading*.
2. **Run-Time Polymorphism (Dynamic / Late Binding):** Resolved during application execution via dynamic method dispatch. Exemplified by *method overriding*.

2. Method Overloading Mechanics and Signature Disambiguation

Method overloading occurs when a single class contains two or more methods sharing the exact same identifier (name), but distinct parameter signatures. In Java, a **method signature** is composed strictly of:

- The method identifier (name).
- The number of formal parameters.
- The data types of the parameters.
- The sequential order of the parameter types.

Crucial Rule on Return Types: In Java, a method's return type is *not* part of its signature. Consequently, defining two methods with identical names and identical parameter types but different return types results in a fatal compile-time error (`duplicate method error`). This design prevents invocation ambiguity: if a method call ignores the return value (e.g., `computeArea(5.0);`), the compiler cannot determine which method to invoke based solely on the return type.

During compilation, the Java compiler matches the invocation arguments against declared signatures using a strict three-phase overload resolution process:

1. Exact match without subtyping or boxing.
2. Match using automatic primitive type promotion (Widening Primitive Conversion: `byte` → `short` → `int` → `long` → `float` → `double`).
3. Match using Autoboxing/Unboxing and variable-arity (varargs) parameters.

3. Constructor Overloading and Constructor Chaining

A **constructor** is a specialized member block invoked automatically upon object instantiation to allocate resources and establish invariant object state. Constructors possess no explicit return type (not even `void`) and bear a name identical to the enclosing class.

Constructor Overloading enables a class to define multiple constructors with diverse parameter lists, catering to varied initialization scenarios. For instance, a geometric entity may be constructed with zero parameters (default state), one parameter (uniform cube/sphere), two parameters (cylinder: radius and height), or three parameters (rectangular cuboid: length, breadth, and height).

To eliminate code redundancy across overloaded constructors, Java provides **constructor chaining** using the `this(...)` invocation. When utilized, `this(...)` must appear as the very first executable statement in the constructor body, delegating shared initialization logic to a centralized constructor.

4. Industrial and Practical Relevance

Method and constructor overloading are extensively leveraged throughout the Java Standard Library. Notable examples include `java.lang.Math` (where methods like `abs()`, `max()`, and `min()` are overloaded for `int`, `long`, `float`, and `double`) and `java.io.PrintStream` (where `println()` is overloaded for every primitive and reference type). Overloading provides consistent, intuitive APIs while upholding strict compile-time type safety.

■ ALGORITHM

1. Start the program.
2. Define class `GeometricSolid` with fields: `shapeType` (String), `dimension1`, `dimension2`, `dimension3` (double).
3. Implement Constructor 1: `GeometricSolid()` default zero-parameter constructor setting `shapeType` to 'Undefined / Point Solid' and dimensions to 0.0.
4. Implement Constructor 2: `GeometricSolid(double side)` single-parameter constructor setting `shapeType` to 'Uniform Cube' and all dimensions to side.
5. Implement Constructor 3: `GeometricSolid(double radius, double height)` two-parameter constructor setting `shapeType` to 'Circular Cylinder' with `dimension1=radius` and `dimension2=height`.
6. Implement Constructor 4: `GeometricSolid(double length, double breadth, double height)` three-parameter constructor for 'Rectangular Cuboid'.
7. Implement `calculateVolume()` to compute volume based on `shapeType` using geometric formulas, and `displaySolidSpecs()` to print parameters.
8. Define class `AreaCalculator` with overloaded `computeArea()` methods:
9. a. `computeArea(double side)` for Square: return `side * side`.
10. b. `computeArea(float radius)` for Circle: return `PI * radius * radius`.
11. c. `computeArea(double length, double breadth)` for Rectangle: return `length * breadth`.
12. d. `computeArea(double base, double height, boolean isTriangle)` for Triangle: return `0.5 * base * height`.
13. e. `computeArea(double a, double b, double h, String label)` for Trapezoid: return `0.5 * (a + b) * h`.

14. In `main()`, instantiate `GeometricSolid` objects using each overloaded constructor and invoke `displaySolidSpecs()`.
15. Instantiate `AreaCalculator` and invoke each overloaded `computeArea()` method with corresponding arguments; display computed areas.
16. Stop.

■ PROGRAM

```
// Experiment 2: Implement method overloading and constructor overloading.
// Subject: SBL - OOP Java
// Student: Anish Vyapari | Roll No: 25CA1012 | PRN: DY25ENGU0AIM012

class GeometricSolid {
    private String shapeType;
    private double dimension1;
    private double dimension2;
    private double dimension3;

    // 1. CONSTRUCTOR OVERLOADING

    // Default constructor (Zero parameters)
    public GeometricSolid() {
        this.shapeType = "Undefined / Point Solid";
        this.dimension1 = 0.0;
        this.dimension2 = 0.0;
        this.dimension3 = 0.0;
        System.out.println("[Constructor 1 Invoked]: Default zero-parameter constructor.");
    }

    // Single-parameter constructor (Cube / Sphere)
    public GeometricSolid(double side) {
        this.shapeType = "Uniform Cube";
        this.dimension1 = side;
        this.dimension2 = side;
        this.dimension3 = side;
        System.out.println("[Constructor 2 Invoked]: Single-parameter constructor (Side = " + side
+ ").");
    }

    // Two-parameter constructor (Cylinder: radius and height)
    public GeometricSolid(double radius, double height) {
        this.shapeType = "Circular Cylinder";
        this.dimension1 = radius;
        this.dimension2 = height;
        this.dimension3 = 0.0;
        System.out.println("[Constructor 3 Invoked]: Two-parameter constructor (Radius = " +
radius + ", Height = " + height + ").");
    }

    // Three-parameter constructor (Rectangular Cuboid: length, breadth, height)
    public GeometricSolid(double length, double breadth, double height) {
        this.shapeType = "Rectangular Cuboid";
        this.dimension1 = length;
        this.dimension2 = breadth;
        this.dimension3 = height;
        System.out.println("[Constructor 4 Invoked]: Three-parameter constructor (L = " + length +
", B = " + breadth + ", H = " + height + ").");
    }

    // Calculate volume based on the initialized solid
    public double calculateVolume() {
```

```

        if (shapeType.equals("Uniform Cube")) {
            return dimension1 * dimension1 * dimension1;
        } else if (shapeType.equals("Circular Cylinder")) {
            return Math.PI * dimension1 * dimension1 * dimension2;
        } else if (shapeType.equals("Rectangular Cuboid")) {
            return dimension1 * dimension2 * dimension3;
        } else {
            return 0.0;
        }
    }

    public void displaySolidSpecs() {
        System.out.printf("  Type: %-22s | Volume: %10.2f cu units\n", shapeType,
            calculateVolume());
    }
}

class AreaCalculator {
    // 2. METHOD OVERLOADING

    // Overloaded Method 1: Area of a Square (1 parameter - double)
    public double computeArea(double side) {
        System.out.println("→ Executing: computeArea(double side) [Square]");
        return side * side;
    }

    // Overloaded Method 2: Area of a Circle (1 parameter - float)
    public double computeArea(float radius) {
        System.out.println("→ Executing: computeArea(float radius) [Circle]");
        return Math.PI * radius * radius;
    }

    // Overloaded Method 3: Area of a Rectangle (2 parameters - double, double)
    public double computeArea(double length, double breadth) {
        System.out.println("→ Executing: computeArea(double length, double breadth)
[Rectangle]");
        return length * breadth;
    }

    // Overloaded Method 4: Area of a Triangle (3 parameters - double, double, boolean flag)
    public double computeArea(double base, double height, boolean isTriangle) {
        System.out.println("→ Executing: computeArea(double base, double height, boolean)
[Triangle]");
        if (isTriangle) {
            return 0.5 * base * height;
        }
        return base * height;
    }

    // Overloaded Method 5: Area of a Trapezoid (4 parameters - double, double, double, String)
    public double computeArea(double sideA, double sideB, double height, String label) {
        System.out.println("→ Executing: computeArea(sideA, sideB, h, label) [" + label + "]");
        return 0.5 * (sideA + sideB) * height;
    }
}

public class Exp20overloading {
    public static void main(String[] args) {
        System.out.println("=== RAIT Nerul - SBL OOP Java Lab: Experiment 2 ===");
        System.out.println("Candidate: Anish Vyapari | Roll No: 25CA1012");
        System.out.println("Topic: Method Overloading and Constructor Overloading\n");

        System.out.println("=====");
        System.out.println("PART A: DEMONSTRATION OF CONSTRUCTOR OVERLOADING");
    }
}

```

```

        System.out.println("=====");

        GeometricSolid solid1 = new GeometricSolid();
        solid1.displaySolidSpecs();

        GeometricSolid solid2 = new GeometricSolid(6.0);
        solid2.displaySolidSpecs();

        GeometricSolid solid3 = new GeometricSolid(4.0, 10.0);
        solid3.displaySolidSpecs();

        GeometricSolid solid4 = new GeometricSolid(8.0, 5.0, 3.0);
        solid4.displaySolidSpecs();

        System.out.println("\n=====");
        System.out.println("PART B: DEMONSTRATION OF METHOD OVERLOADING");
        System.out.println("=====");

        AreaCalculator calc = new AreaCalculator();

        double squareArea = calc.computeArea(7.5);
        System.out.printf("    Calculated Square Area      : %.2f sq units\n\n", squareArea);

        double circleArea = calc.computeArea(5.0f);
        System.out.printf("    Calculated Circle Area      : %.2f sq units\n\n", circleArea);

        double rectArea = calc.computeArea(12.0, 6.5);
        System.out.printf("    Calculated Rectangle Area : %.2f sq units\n\n", rectArea);

        double triArea = calc.computeArea(10.0, 8.0, true);
        System.out.printf("    Calculated Triangle Area   : %.2f sq units\n\n", triArea);

        double trapArea = calc.computeArea(10.0, 14.0, 6.0, "Trapezoid");
        System.out.printf("    Calculated Trapezoid Area  : %.2f sq units\n", trapArea);

        System.out.println("=====");
    }
}

```

■ CONCLUSION

Successfully implemented compile-time polymorphism in Java through method overloading and constructor overloading. Observed how the Java compiler resolves target methods at compile time using signature parameter signatures and type promotion rules, enabling clean, intuitive, and versatile class interfaces.

EXPERIMENT 3

Build a program on 1D and 2D arrays.

■ AIM

To build and execute a Java program demonstrating array declaration, memory allocation, initialization, sequential traversal, linear searching, sorting, and matrix transformations using one-dimensional (1D) and two-dimensional (2D) arrays.

■ OBJECTIVE

- Understand array memory allocation in Java as first-class heap objects with dynamic bounds checking.
- Perform sequential traversal, statistical aggregation (sum, mean, min, max), and linear searching on 1D arrays.
- Implement in-place sorting (Bubble Sort algorithm) to order elements in ascending numerical sequence.
- Construct and manipulate 2D arrays (matrices) to execute matrix addition, matrix transposition, and diagonal trace analysis.

■ SOFTWARE REQUIRED

Java Development Kit (JDK 21 LTS), Visual Studio Code / Eclipse IDE, Windows Terminal.

■ THEORY

1. Architectural Representation of Arrays in Java

In Java, an **array** is a fixed-size, homogeneous data structure that stores primitive values or object references in contiguous memory locations. Unlike low-level languages such as C and C++ where array identifiers act merely as pointers to raw memory buffers, an array in Java is a **true object** inheriting directly from `java.lang.Object`. Array objects reside dynamically in the Java Heap memory, while the array reference variable resides in stack memory.

Key architectural properties of Java arrays include:

- **Type Safety:** Arrays are strongly typed. An array declared as `int[]` can only accept 32-bit signed integers.
- **Invariable Length:** Once instantiated via `new`, an array's length is immutable and stored in a public final attribute `length` (e.g., `arr.length`).
- **Automatic Initialization:** Array components on the heap are automatically initialized to default values upon allocation (`0` for numeric primitives, `false` for boolean, `\u0000` for char, and `null` for reference types).
- **Runtime Bounds Checking:** Every array indexing operation is verified at runtime by the JVM. Attempting to access an index outside `[0, length - 1]` throws an unchecked `ArrayIndexOutOfBoundsException`, protecting memory integrity against buffer overflow attacks.

2. One-Dimensional (1D) Array Lifecycle and Operations

A 1D array represents a single row of indexed elements. Its lifecycle involves:

1. **Declaration:** `int[] scores;` (declares a reference on the stack; no memory allocated).
2. **Instantiation:** `scores = new int[10];` (allocates heap space for 10 integer slots).

3. **Initialization:** Direct assignment or array literal notation (`int[] scores = {45, 88, 72, 95...};`).

Fundamental algorithms executed on 1D arrays include:

- **Sequential Traversal:** Iterating through indices using traditional for loops or enhanced for-each loops to aggregate statistics such as summation, mean calculation, and extrema detection (min/max).
- **Linear Search:** Scanning elements sequentially from index 0 to $N-1$ until a target key is located, exhibiting $O(N)$ time complexity.
- **Bubble Sort:** An elementary comparison-based sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. Across $N-1$ passes, unsorted elements "bubble" to their correct positions, yielding an $O(N^2)$ time complexity.

3. Two-Dimensional (2D) Arrays: Array-of-Arrays Concept

Java does not allocate multidimensional arrays as contiguous flat memory blocks. Instead, a two-dimensional array is implemented architecturally as an **array of arrays**. In `int[][] matrix = new int[3][3];`:

- `matrix` points to a top-level array of length 3 residing on the heap.
- Each of the 3 slots in the top-level array holds a reference to a distinct 1D array of 3 integer elements.
- This architectural decoupling enables Java to support *ragged (jagged) arrays*, where different rows possess differing column lengths.

Standard matrix mathematical operations include:

- **Matrix Addition:** For two $M \times N$ matrices A and B , their sum C is computed element-wise as $C[i][j] = A[i][j] + B[i][j]$.
- **Matrix Transposition:** Reflecting a matrix across its main diagonal, transforming row indices into column indices such that $A^T[j][i] = A[i][j]$.
- **Matrix Trace:** The sum of elements along the principal diagonal ($A[i][i]$ where $i = j$) for a square matrix.

4. Industrial Applications

Arrays underpin high-performance computing, digital signal processing, tabular database query caching, image pixel manipulation (represented as 2D/3D color buffers), and graph representation via adjacency matrices.

■ ALGORITHM

1. Start the program.
2. In `demonstrate1DArray()`:
3. a. Declare and initialize 1D array `scores` with 10 integers: {45, 88, 72, 95, 60, 34, 82, 91, 53, 76}.
4. b. Print original array elements and `scores.length`.
5. c. Initialize `sum = 0`, `min = scores[0]`, `max = scores[0]`. Traverse array using for-each loop; accumulate `sum`, update `min` if `element < min`, update `max` if `element > max`.
6. d. Compute `average = (double) sum / scores.length`; display `sum`, `average`, `min`, and `max`.
7. e. Perform Linear Search for `searchKey = 82`: iterate index `i` from 0 to `length - 1`; if `scores[i] == searchKey`, record `foundIndex = i` and break.
8. f. Perform Bubble Sort: run outer loop `i` from 0 to `n - 2`, inner loop `j` from 0 to `n - i - 2`; if `scores[j] > scores[j+1]`, swap elements using temporary variable.
9. g. Display sorted 1D array in ascending order.

10. In demonstrate2DArray():
11. a. Initialize 3×3 Matrix A and Matrix B with integer coefficients.
12. b. Display Matrix A and Matrix B in formatted tabular layout using helper displayMatrix().
13. c. Compute matrix sum into sumMatrix[3][3] using nested loops: $\text{sumMatrix}[i][j] = \text{matrixA}[i][j] + \text{matrixB}[i][j]$.
14. d. Compute transpose of Matrix A into transposeA[3][3]: $\text{transposeA}[j][i] = \text{matrixA}[i][j]$.
15. e. Compute row-wise sums of Matrix A and calculate principal diagonal sum (trace) where $i == j$.
16. f. Display matrix sum, transpose matrix, row sums, and trace.
17. In main(), invoke demonstrate1DArray() and demonstrate2DArray().
18. Stop.

■ PROGRAM

```
// Experiment 3: Build a program on 1D and 2D arrays.
// Subject: SBL - OOP Java
// Student: Anish Vyapari | Roll No: 25CA1012 | PRN: DY25ENGU0AIM012

public class Exp3Arrays {

    // --- 1D ARRAY OPERATIONS ---

    public static void demonstrate1DArray() {
        System.out.println("=====");
        System.out.println("PART 1: ONE-DIMENSIONAL (1D) ARRAY OPERATIONS");
        System.out.println("=====");

        // 1. Declaration and Initialization
        int[] scores = {45, 88, 72, 95, 60, 34, 82, 91, 53, 76};
        System.out.print("Original 1D Array Elements: [ ");
        for (int i = 0; i < scores.length; i++) {
            System.out.print(scores[i] + (i < scores.length - 1 ? ", " : " "));
        }
        System.out.println("]");
        System.out.println("Array Length: " + scores.length);

        // 2. Statistical Computations (Min, Max, Sum, Average)
        int sum = 0;
        int min = scores[0];
        int max = scores[0];

        for (int val : scores) {
            sum += val;
            if (val < min) min = val;
            if (val > max) max = val;
        }

        double average = (double) sum / scores.length;
        System.out.println("Sum of Elements      : " + sum);
        System.out.printf("Mean / Average Value : %.2f\n", average);
        System.out.println("Minimum Element     : " + min);
        System.out.println("Maximum Element      : " + max);

        // 3. Linear Search Operation
        int searchKey = 82;
        int foundIndex = -1;
        for (int i = 0; i < scores.length; i++) {
            if (scores[i] == searchKey) {
                foundIndex = i;
            }
        }
    }
}
```

```

        break;
    }
}
if (foundIndex  $\neq$  -1) {
    System.out.println("Linear Search: Key " + searchKey + " found at Index " +
foundIndex);
} else {
    System.out.println("Linear Search: Key " + searchKey + " not present in array.");
}

// 4. Sorting 1D Array (Bubble Sort Implementation)
int n = scores.length;
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 1; j++) {
        if (scores[j] > scores[j + 1]) {
            int temp = scores[j];
            scores[j] = scores[j + 1];
            scores[j + 1] = temp;
        }
    }
}

System.out.print("Sorted 1D Array (Ascending): [ ");
for (int i = 0; i < scores.length; i++) {
    System.out.print(scores[i] + (i < scores.length - 1 ? ", " : " "));
}
System.out.println("]");
}

// --- 2D ARRAY OPERATIONS ---

public static void displayMatrix(String title, int[][] matrix) {
    System.out.println(title + ":");
    for (int i = 0; i < matrix.length; i++) {
        System.out.print(" | ");
        for (int j = 0; j < matrix[i].length; j++) {
            System.out.printf("%4d ", matrix[i][j]);
        }
        System.out.println("|");
    }
}

public static void demonstrate2DArray() {
    System.out.println("\n=====");
    System.out.println("PART 2: TWO-DIMENSIONAL (2D) ARRAY OPERATIONS");
    System.out.println("=====");

    // Matrix A (3x3)
    int[][] matrixA = {
        {3, 8, 2},
        {4, 6, 9},
        {1, 5, 7}
    };

    // Matrix B (3x3)
    int[][] matrixB = {
        {5, 2, 7},
        {8, 1, 3},
        {6, 4, 2}
    };

    int rows = matrixA.length;
    int cols = matrixA[0].length;

```

```

displayMatrix("Matrix A (3x3)", matrixA);
displayMatrix("Matrix B (3x3)", matrixB);

// 1. Matrix Addition: C = A + B
int[][] sumMatrix = new int[rows][cols];
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        sumMatrix[i][j] = matrixA[i][j] + matrixB[i][j];
    }
}
displayMatrix("Matrix Sum (Matrix A + Matrix B)", sumMatrix);

// 2. Matrix Transpose of Matrix A (Rows become columns)
int[][] transposeA = new int[cols][rows];
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        transposeA[j][i] = matrixA[i][j];
    }
}
displayMatrix("Transpose of Matrix A (A^T)", transposeA);

// 3. Row-wise and Column-wise Sum Calculation
System.out.println("\nRow and Column Diagnostics of Matrix A:");
for (int i = 0; i < rows; i++) {
    int rowSum = 0;
    for (int j = 0; j < cols; j++) {
        rowSum += matrixA[i][j];
    }
    System.out.println("  Row " + i + " Sum: " + rowSum);
}

// Trace of Matrix A (Principal diagonal sum)
int trace = 0;
for (int i = 0; i < rows; i++) {
    trace += matrixA[i][i];
}
System.out.println("  Principal Diagonal Sum (Trace): " + trace);
}

public static void main(String[] args) {
    System.out.println("=== RAIT Nerul - SBL OOP Java Lab: Experiment 3 ===");
    System.out.println("Candidate: Anish Vyapari | Roll No: 25CA1012");
    System.out.println("Topic: One-Dimensional (1D) and Two-Dimensional (2D) Arrays\n");

    demonstrate1DArray();
    demonstrate2DArray();
    System.out.println("=====");
}
}

```

■ CONCLUSION

Successfully designed and executed a Java program manipulating 1D and 2D arrays. Verified linear search, statistical aggregation, and bubble sorting on 1D arrays, as well as matrix addition, transposition, and trace diagnostics on 2D arrays-of-arrays in the Java heap.

EXPERIMENT 4

Implementation of string manipulation functions and write a program to check whether it is palindrome or not.

■ AIM

To implement standard string manipulation methods using Java's String and StringBuilder classes, analyze String immutability, and write an algorithm to verify whether a given string is a palindrome.

■ OBJECTIVE

- Understand the architectural immutability of the java.lang.String class and the String Constant Pool (SCP).
- Apply string manipulation methods: length(), charAt(), substring(), toUpperCase(), toLowerCase(), indexOf(), replace(), concat(), trim(), and equals().
- Differentiate between reference identity equality (==) and value/content equality (.equals()).
- Implement palindrome verification using both a two-pointer algorithmic scan and StringBuilder reversal.

■ SOFTWARE REQUIRED

Java Development Kit (JDK 21 LTS), Visual Studio Code / Eclipse IDE, Windows Terminal.

■ THEORY

1. Architectural Foundations of the String Class in Java

In Java, character sequences are encapsulated within the `java.lang.String` class. Unlike primitive data types or arrays of characters in C, a Java String is an immutable object. The internal representation stores characters in an encapsulated byte array (compact strings optimization in modern JDKs) accompanied by an encoding coder flag.

2. The Principle of String Immutability and the String Constant Pool (SCP)

Immutability dictates that once a `String` instance is instantiated on the heap, its character sequence can never be modified. Any method that appears to transform a string (such as `replace()`, `toUpperCase()`, or concatenation) does not alter the original object; instead, it instantiates and returns an entirely new `String` object with the updated content.

The primary rationale for String immutability in Java includes:

- **Memory Optimization via String Constant Pool (SCP):** The SCP is a dedicated cache region within the Java Heap. When a string literal is created (e.g., `String s = "Java";`), the JVM checks the SCP. If an identical sequence already exists, the existing reference is shared rather than allocating redundant memory. If strings were mutable, altering a string through one reference would corrupt the value seen by all other references sharing that pooled literal.
- **Thread Safety:** Because String objects cannot be modified after construction, they are inherently immutable and safe to share across concurrent threads without synchronization overhead.
- **Security:** Strings are extensively utilized for class loading paths, network sockets, database URLs, and authentication credentials. Immutability ensures that sensitive strings cannot be tampered with after initial validation.
- **HashCode Caching:** The hashcode of a String is computed once and cached, drastically accelerating lookup performance in hash-based collections like `HashMap` and `HashSet`.

3. Reference Equality (==) vs Content Equality (.equals())

A frequent pitfall in Java programming is confusing `=` with `equals()` :

- `=` Operator: Evaluates **reference identity**—testing whether two variables point to the exact same memory address on the heap.
- `.equals()` Method: Overridden in the `String` class to evaluate **content equality**—comparing character by character to determine whether two strings represent identical text.
- `.equalsIgnoreCase()` : Compares character sequences ignoring differences in ASCII/Unicode casing.

4. Mutable String Processing: StringBuilder and StringBuffer

When high-frequency string concatenations or mutations occur, using immutable Strings creates excessive intermediate garbage objects, burdening the Garbage Collector. Java provides two mutable companion classes:

- `StringBuilder` : Introduced in Java 5; provides high-performance, non-synchronized, mutable character buffers optimal for single-threaded processing.
- `StringBuffer` : Legacy synchronized, thread-safe mutable alternative with synchronization overhead.

5. Palindrome Verification Mechanics

A **palindrome** is a sequence of characters that reads identically forward and backward (e.g., "Radar", "Madam", "12321"). Algorithmic verification can be executed via two methodologies:

1. **Two-Pointer Inward Scan:** Two indices, `left = 0` and `right = length - 1`, move toward the center. Characters at each position are compared after case and non-alphanumeric normalization. If any mismatch occurs, the string is not a palindrome. This approach operates in $O(N)$ time and $O(1)$ auxiliary space.
2. **StringBuilder Reversal:** Normalizing the string, reversing it using `new StringBuilder(str).reverse().toString()`, and comparing the reversed string with the normalized input using `.equals()`.

■ ALGORITHM

1. Start the program.
2. In `demonstrateStringFunctions()`:
3. a. Initialize string literal `sampleStr = ' Object Oriented Programming in Java '`.
4. b. Apply `length()` and display total character count.
5. c. Apply `trim()` to strip leading/trailing whitespace and store in `trimmed`.
6. d. Extract character at index 7 using `charAt(7)`.
7. e. Extract substrings using `substring(0, 15)` and `substring(16)`.
8. f. Convert to upper and lower case using `toUpperCase()` and `toLowerCase()`.
9. g. Find character and substring indices using `indexOf('Java')` and `indexOf('e')`.
10. h. Replace occurrences of 'Java' with 'Modern Java 21' using `replace()`.
11. i. Concatenate string using `concat(' - RAIT Lab')`.
12. j. Validate prefixes and suffixes using `startsWith('Object')` and `endsWith('Java')`.
13. k. Compare strings 'JavaProgramming' and 'javaprogramming' using `equals()` and `equalsIgnoreCase()`.
14. In `isPalindromeAlgorithmic(input)`:
15. a. Clean input by removing non-alphanumeric characters and converting to lowercase: `input.replaceAll('[^a-zA-Z0-9]', '').toLowerCase()`.

16. b. Initialize left = 0 and right = cleaned.length() - 1.
17. c. While left < right: if cleaned.charAt(left) != cleaned.charAt(right), return false; increment left, decrement right.
18. d. Return true if loop terminates without mismatch.
19. In isPalindromeBuilder(input):
20. a. Clean input identically.
21. b. Instantiate StringBuilder with cleaned string, invoke reverse().toString(), and compare with cleaned using equals().
22. In demonstratePalindrome():
23. a. Define test suite: 'Radar', 'Madam', 'A man a plan a canal Panama', 'Was it a car or a cat I saw', 'Computer Science', '12321', 'ObjectOriented'.
24. b. Iterate through test suite, invoke both palindrome methods, and display comparison table.
25. In main(), invoke demonstrateStringFunctions() and demonstratePalindrome().
26. Stop.

■ PROGRAM

```
// Experiment 4: Implementation of string manipulation functions and write a program to check
// whether it is palindrome or not.
// Subject: SBL - OOP Java
// Student: Anish Vyapari | Roll No: 25CA1012 | PRN: DY25ENGU0AIM012

public class Exp4Strings {

    public static void demonstrateStringFunctions() {
        System.out.println("=====");
        System.out.println("PART 1: STRING MANIPULATION FUNCTIONS");
        System.out.println("=====");

        String sampleStr = " Object Oriented Programming in Java ";
        System.out.println("Original String literal: \"" + sampleStr + "\"");

        // 1. length()
        System.out.println("1. Length of string: " + sampleStr.length());

        // 2. trim()
        String trimmed = sampleStr.trim();
        System.out.println("2. After trim(): \"" + trimmed + "\" (Length: " + trimmed.length() +
            ")");

        // 3. charAt()
        System.out.println("3. Character at index 7: '" + trimmed.charAt(7) + "'");

        // 4. substring()
        String sub1 = trimmed.substring(0, 15);
        String sub2 = trimmed.substring(16);
        System.out.println("4. Substring (0 to 15): \"" + sub1 + "\"");
        System.out.println("   Substring (from 16): \"" + sub2 + "\"");

        // 5. toUpperCase() and toLowerCase()
        System.out.println("5. toUpperCase(): \"" + trimmed.toUpperCase() + "\"");
        System.out.println("   toLowerCase(): \"" + trimmed.toLowerCase() + "\"");

        // 6. indexOf() and lastIndexOf()
        System.out.println("6. Index of 'Java': " + trimmed.indexOf("Java"));
        System.out.println("   Index of character 'e': " + trimmed.indexOf('e'));
```

```

// 7. replace()
String replaced = trimmed.replace("Java", "Modern Java 21");
System.out.println("7. replace(\"Java\", \"Modern Java 21\"): \"" + replaced + "\"");

// 8. concat()
String concatenated = trimmed.concat(" - RAIT Lab");
System.out.println("8. concat(\" - RAIT Lab\"): \"" + concatenated + "\"");

// 9. startsWith() and endsWith()
System.out.println("9. startsWith(\"Object\"): " + trimmed.startsWith("Object"));
System.out.println("    endsWith(\"Java\"): " + trimmed.endsWith("Java"));

// 10. equals() vs equalsIgnoreCase()
String strA = "JavaProgramming";
String strB = "javaprogramming";
System.out.println("10. Comparing \"" + strA + "\" with \"" + strB + "\"");
System.out.println("    equals()           : " + strA.equals(strB));
System.out.println("    equalsIgnoreCase() : " + strA.equalsIgnoreCase(strB));
}

// Two-pointer algorithmic palindrome checker
public static boolean isPalindromeAlgorithmic(String input) {
    if (input == null) return false;
    // Clean non-alphanumeric and convert to lowercase
    String cleaned = input.replaceAll("[^a-zA-Z0-9]", "").toLowerCase();
    int left = 0;
    int right = cleaned.length() - 1;

    while (left < right) {
        if (cleaned.charAt(left) != cleaned.charAt(right)) {
            return false;
        }
        left++;
        right--;
    }
    return true;
}

// StringBuilder reverse method palindrome checker
public static boolean isPalindromeBuilder(String input) {
    if (input == null) return false;
    String cleaned = input.replaceAll("[^a-zA-Z0-9]", "").toLowerCase();
    String reversed = new StringBuilder(cleaned).reverse().toString();
    return cleaned.equals(reversed);
}

public static void testPalindrome(String phrase) {
    boolean res1 = isPalindromeAlgorithmic(phrase);
    boolean res2 = isPalindromeBuilder(phrase);
    System.out.printf("Input: \"%-30s\" | Two-Pointer: %-5s | StringBuilder: %-5s | Result: %s\n",
        phrase, res1, res2, (res1 ? "PALINDROME" : "NOT A PALINDROME"));
}

public static void demonstratePalindrome() {
    System.out.println("\n=====");
    System.out.println("PART 2: PALINDROME STRING VERIFICATION");
    System.out.println("=====");

    String[] testStrings = {
        "Radar",
        "Madam",
        "A man a plan a canal Panama",
    }

```

```

        "Was it a car or a cat I saw",
        "Computer Science",
        "12321",
        "ObjectOriented"
    };

    for (String test : testStrings) {
        testPalindrome(test);
    }
}

public static void main(String[] args) {
    System.out.println("≡≡≡ RAIT Nerul - SBL OOP Java Lab: Experiment 4 ≡≡≡");
    System.out.println("Candidate: Anish Vyapari | Roll No: 25CA1012");
    System.out.println("Topic: String Manipulation Functions and Palindrome Verification\n");

    demonstrateStringFunctions();
    demonstratePalindrome();
    System.out.println("=====");
}
}

```

■ CONCLUSION

Successfully implemented and analyzed string manipulation functions in Java. Validated the architectural mechanics of String immutability, the String Constant Pool, reference versus value equality, and executed palindrome verification using both algorithmic two-pointer scanning and StringBuilder reversal.

EXPERIMENT 5

Implement a single and multilevel inheritance (use super keyword).

■ AIM

To implement single-level and multilevel inheritance in Java, demonstrating code reusability, specialization, and invoking superclass constructors and overridden methods using the super keyword.

■ OBJECTIVE

- Understand the core OOP principle of Inheritance and the IS-A conceptual relationship.
- Implement Single Inheritance (two-tier hierarchy) and Multilevel Inheritance (three-tier hierarchy).
- Master the super keyword for constructor chaining (super()) and accessing overridden methods (super.method()).
- Analyze access modifier semantics (protected, private, public) across inheritance boundaries.

■ SOFTWARE REQUIRED

Java Development Kit (JDK 21 LTS), Visual Studio Code / Eclipse IDE, Windows Terminal.

■ THEORY

1. Foundations of Inheritance in Object-Oriented Programming

Inheritance is a foundational pillar of object-oriented programming that allows a new class (known as the *subclass*, *derived class*, or *child class*) to inherit properties (fields) and behaviors (methods) from an existing class (known as the *superclass*, *base class*, or *parent class*). Inheritance models the fundamental **IS-A relationship** (e.g., an `Employee` IS-A `Person`, a `Manager` IS-A `Employee`).

The primary architectural benefits of inheritance include:

- **Code Reusability:** Common attributes and routines are defined once in a generalized superclass and automatically propagated to specialized subclasses, eliminating redundant logic.
- **Method Overriding & Runtime Polymorphism:** Subclasses can provide specific implementations of methods declared in superclasses.
- **Extensibility:** New functionality can be added to existing software frameworks without modifying established, tested code, upholding the Open/Closed Principle (OCP).

2. Forms of Inheritance in Java

Java supports several inheritance topologies through the `extends` keyword:

1. **Single Inheritance:** A single subclass derives directly from a single superclass (e.g., `class Employee extends Person`).
2. **Multilevel Inheritance:** A subclass derives from an intermediate class, which in turn derives from a base class, establishing a multi-tier ancestor chain (e.g., `class TechnicalProjectLead extends Employee extends Person`).
3. **Hierarchical Inheritance:** Multiple subclasses derive from a common superclass.

Multiple Inheritance Limitation: Java explicitly disallows multiple inheritance using classes (i.e., a class cannot extend more than one direct parent class) to prevent architectural ambiguity known as the *Diamond Problem*. Multiple inheritance of type is instead achieved via interfaces.

3. The super Keyword: Architecture and Usage

The `super` keyword in Java is a reference variable used by subclasses to refer directly to their immediate parent class object. It serves three distinct functions:

- **Invoking Superclass Constructors (`super(...)`)**: When an object of a derived class is instantiated, the superclass constructor must execute first to initialize base-class state. Invoking `super(arg1, arg2...)` calls the matching parent constructor. This statement *must be the very first executable line* in the subclass constructor. If omitted, the Java compiler automatically inserts an implicit zero-argument call `super();`.
- **Invoking Overridden Superclass Methods (`super.methodName()`)**: When a subclass overrides a parent method, the parent's original implementation can still be accessed from within the subclass using `super.methodName()`, facilitating incremental behavioral extension rather than complete replacement.
- **Accessing Hidden Parent Fields (`super.fieldName`)**: Disambiguates fields when a subclass declares a field with the exact same identifier as a field in its superclass.

4. Access Control in Inheritance

Inheritance respects Java's encapsulation boundaries:

- `private` members of a superclass are not accessible directly within subclasses; they can only be accessed via public or protected getters/setters.
- `protected` members are accessible by subclasses anywhere, as well as by other classes within the same package.
- `public` members are fully inherited and accessible everywhere.

■ ALGORITHM

1. Start the program.
2. Define base class Person with protected attributes: name (String), age (int), bloodGroup (String).
3. Implement Person constructor Person(name, age, bloodGroup) initializing fields with diagnostic print statement.
4. Implement displayProfile() in Person to print personal demographics.
5. Define subclass Employee extending Person (Single Inheritance):
6. a. Declare protected attributes: employeeId (String), department (String), baseSalary (double).
7. b. Implement Employee constructor invoking super(name, age, bloodGroup) as first statement, then initialize employeeId, department, baseSalary.
8. c. Override displayProfile() invoking super.displayProfile() followed by employment metrics.
9. d. Implement calculateNetSalary() computing HRA (20%), DA (10%), PF deduction (12%).
10. Define subclass TechnicalProjectLead extending Employee (Multilevel Inheritance):
11. a. Declare private attributes: projectName (String), teamSize (int), projectAllowance (double).
12. b. Implement TechnicalProjectLead constructor invoking super(name, age, bloodGroup, employeeId, department, baseSalary) to trigger chained parent initialization.
13. c. Initialize projectName, teamSize, projectAllowance.
14. d. Override displayProfile() invoking super.displayProfile() followed by project leadership specs.
15. e. Implement computeLeadershipPackage() adding projectAllowance to inherited calculateNetSalary().
16. In main(), demonstrate Single Inheritance by instantiating Employee object and invoking displayProfile() and calculateNetSalary().

17. Demonstrate Multilevel Inheritance by instantiating TechnicalProjectLead object and invoking displayProfile() and computeLeadershipPackage().

18. Stop.

■ PROGRAM

```
// Experiment 5: Implement a single and multilevel inheritance (use super keyword).
// Subject: SBL - OOP Java
// Student: Anish Vyapari | Roll No: 25CA1012 | PRN: DY25ENGU0AIM012

// Base Class: Person (Root of the inheritance hierarchy)
class Person {
    protected String name;
    protected int age;
    protected String bloodGroup;

    // Base class constructor
    public Person(String name, int age, String bloodGroup) {
        this.name = name;
        this.age = age;
        this.bloodGroup = bloodGroup;
        System.out.println(" [Person Constructor]: Initialized personal profile for " + name);
    }

    public void displayProfile() {
        System.out.println(" Personal Info → Name: " + name + " | Age: " + age + " | Blood
Group: " + bloodGroup);
    }
}

// Level 1 Subclass: Employee (Demonstrating SINGLE INHERITANCE: Employee extends Person)
class Employee extends Person {
    protected String employeeId;
    protected String department;
    protected double baseSalary;

    // Subclass constructor invoking superclass constructor via super()
    public Employee(String name, int age, String bloodGroup, String employeeId, String department,
double baseSalary) {
        super(name, age, bloodGroup); // Invoking Person constructor
        this.employeeId = employeeId;
        this.department = department;
        this.baseSalary = baseSalary;
        System.out.println(" [Employee Constructor]: Initialized employee credentials (ID: " +
employeeId + ")");
    }

    // Overridden method invoking superclass method via super.displayProfile()
    @Override
    public void displayProfile() {
        super.displayProfile(); // Explicitly calling superclass implementation
        System.out.println(" Employment Info → Emp ID: " + employeeId + " | Dept: " + department
+ " | Base Salary: Rs. " + baseSalary);
    }

    public double calculateNetSalary() {
        double hra = 0.20 * baseSalary; // 20% House Rent Allowance
        double da = 0.10 * baseSalary; // 10% Dearness Allowance
        double pf = 0.12 * baseSalary; // 12% Provident Fund
        return baseSalary + hra + da - pf;
    }
}
```

```

// Level 2 Subclass: TechnicalProjectLead (Demonstrating MULTILEVEL INHERITANCE:
TechnicalProjectLead extends Employee)
class TechnicalProjectLead extends Employee {
    private String projectName;
    private int teamSize;
    private double projectAllowance;

    // Derived constructor invoking intermediate parent constructor via super()
    public TechnicalProjectLead(String name, int age, String bloodGroup,
                                String employeeId, String department, double baseSalary,
                                String projectName, int teamSize, double projectAllowance) {
        super(name, age, bloodGroup, employeeId, department, baseSalary); // Invoking Employee
constructor
        this.projectName = projectName;
        this.teamSize = teamSize;
        this.projectAllowance = projectAllowance;
        System.out.println(" [TechnicalProjectLead Constructor]: Initialized leadership role for
" + projectName);
    }

    // Overridden display method
    @Override
    public void displayProfile() {
        super.displayProfile(); // Calls Employee.displayProfile() which calls
Person.displayProfile()
        System.out.println(" Leadership Info → Project: " + projectName + " | Team Size: " +
teamSize + " Engineers | Project Allowance: Rs. " + projectAllowance);
        System.out.printf(" Total Gross Remuneration: Rs. %.2f\n", computeLeadershipPackage());
    }

    public double computeLeadershipPackage() {
        // Accessing inherited logic and adding specific role allowance
        return calculateNetSalary() + projectAllowance;
    }
}

public class Exp5Inheritance {
    public static void main(String[] args) {
        System.out.println("=== RAIT Nerul - SBL OOP Java Lab: Experiment 5 ===");
        System.out.println("Candidate: Anish Vyapari | Roll No: 25CA1012");
        System.out.println("Topic: Single and Multilevel Inheritance with 'super' Keyword\n");

        System.out.println("=====");
        System.out.println("PHASE 1: SINGLE INHERITANCE (Employee IS-A Person)");
        System.out.println("=====");
        System.out.println("Instantiating Employee object (Constructor Chaining):");
        Employee emp = new Employee("Rajesh Kulkarni", 28, "O+", "EMP-2041", "Quality Assurance",
55000.0);
        System.out.println("\nInvoking Employee Profile (demonstrating super.displayProfile()):");
        emp.displayProfile();
        System.out.printf("Calculated Monthly Net Salary: Rs. %.2f\n", emp.calculateNetSalary());

        System.out.println("\n=====");
        System.out.println("PHASE 2: MULTILEVEL INHERITANCE (TechLead IS-A Employee IS-A
Person)");
        System.out.println("=====");
        System.out.println("Instantiating TechnicalProjectLead object (3-Tier Chaining):");
        TechnicalProjectLead lead = new TechnicalProjectLead(
            "Anish Vyapari", 21, "B+",
            "LEAD-1012", "AI & Systems Engineering", 95000.0,
            "Autonomous Edge Core", 8, 25000.0
        );
    }
}

```

```
        System.out.println("\nInvoking Multilevel Display Hierarchy:");
        lead.displayProfile();
        System.out.println("=====");
    }
}
```

■ CONCLUSION

Successfully implemented and validated single and multilevel inheritance hierarchies in Java. Verified constructor chaining across three class tiers using `super()`, accessed overridden superclass methods via `super.displayProfile()`, and confirmed encapsulation and code reuse principles in an enterprise organizational hierarchy.

EXPERIMENT 6

Build an interface demonstrating concept of multiple inheritance.

■ AIM

To build Java interfaces and demonstrate how multiple inheritance of type and behavior is achieved in Java by having a single concrete class implement multiple interfaces.

■ OBJECTIVE

- Understand the concept of interface specification and contract-based design in Java.
- Analyze the Diamond Problem in object-oriented class inheritance and why Java disallows multiple class inheritance.
- Implement multiple interfaces within a single concrete class using the implements keyword.
- Demonstrate polymorphic interface references (upcasting) to achieve loose architectural coupling.

■ SOFTWARE REQUIRED

Java Development Kit (JDK 21 LTS), Visual Studio Code / Eclipse IDE, Windows Terminal.

■ THEORY

1. The Multiple Inheritance Dilemma: The Diamond Problem

In object-oriented system design, **Multiple Inheritance** refers to the ability of a single derived class to inherit directly from more than one parent class. In languages like C++, this model introduces significant architectural peril known as the **Diamond Problem**:

```
      Class A (defines void display())
      /      \
    Class B   Class C (both override display())
      \      /
    Class D (inherits from both B and C)
```

If `Class D` invokes `display()`, the compiler encounters fatal ambiguity: should it execute the implementation inherited from `Class B` or `Class C`? Furthermore, if `Class A` contains mutable instance variables, `Class D` risks inheriting duplicate, conflicting copies of those fields. To ensure linguistic stability, simplicity, and safety, Java's architects deliberately omitted multiple inheritance of classes.

2. The Interface Solution in Java

Java resolves this architectural challenge through the **Interface** mechanism. An interface is a reference type that specifies a pure behavioral contract—declaring *what* a class must do without dictating *how* it does it.

Key architectural properties of Java interfaces include:

- **Method Contracts:** By default, all methods declared in an interface are implicitly `public` and `abstract` (prior to Java 8 default/static methods). They contain no method body and no implementation.
- **Constant Fields:** All fields declared within an interface are implicitly `public static final` (immutable constants). Interfaces cannot declare instance state (instance variables).
- **Multiple Implementation:** A single Java class can implement an unlimited number of interfaces using the `implements` keyword:

```
class TechnicalReport implements Printable, SecureCloudSharable, Auditable { ... }
```

Because interfaces carry method contracts without mutable instance state, the Diamond Problem is eradicated. Even if two interfaces declare the exact same method signature, the implementing class provides a single, unambiguous concrete implementation that satisfies both contracts simultaneously.

3. Polymorphic Interface References and Loose Coupling

Interfaces provide the highest level of abstraction in Java. They enable **Polymorphic Interface Referencing** (Upcasting), wherein an object reference is held by an interface-type variable:

```
Printable pRef = new TechnicalReport(...);
pRef.printDocument(); // Accessible via Printable contract

SecureCloudSharable sRef = new TechnicalReport(...);
sRef.uploadToCloud(...); // Accessible via SecureCloudSharable contract
```

This design decouples caller code from concrete implementations, adhering to the fundamental software engineering principle: *"Program to an interface, not an implementation."* It facilitates interchangeable service providers, modular mocking in unit testing, and dependency inversion.

4. Industrial Applications

Interfaces form the backbone of major enterprise frameworks and the Java Core API, including `java.lang.Comparable`, `java.lang.Runnable`, `java.io.Serializable`, and the entire Java Collections Framework (`List`, `Set`, `Map`).

■ ALGORITHM

1. Start the program.
2. Define interface `Printable` with constant `STANDARD_PAGE_SIZE = 'A4 (210 × 297 mm)'` and abstract methods `printDocument()` and `configurePageLayout(String orientation)`.
3. Define interface `SecureCloudSharable` with constant `ENCRYPTION_STANDARD = 'AES-256 GCM'` and abstract methods `uploadToCloud(String endpoint)` and `verifyChecksum(String hash)`.
4. Define interface `Auditable` with abstract method `logAccessHistory(String user)`.
5. Create concrete class `TechnicalReport` implementing all three interfaces: implements `Printable`, `SecureCloudSharable`, `Auditable`.
6. Declare private fields in `TechnicalReport`: `reportId`, `title`, `author`, `pageCount`, `documentHash`.
7. Implement constructor `TechnicalReport(...)` to initialize fields.
8. Implement `printDocument()` and `configurePageLayout()` fulfilling `Printable` contract.
9. Implement `uploadToCloud()` and `verifyChecksum()` fulfilling `SecureCloudSharable` contract.
10. Implement `logAccessHistory()` fulfilling `Auditable` contract.
11. In `main()`, instantiate a `TechnicalReport` object.
12. Invoke all contract methods directly on the instantiated object.
13. Demonstrate polymorphic interface upcasting: assign object to `Printable` reference and invoke `printDocument()`; assign object to `SecureCloudSharable` reference and invoke `uploadToCloud()`.
14. Stop.

■ PROGRAM

```
// Experiment 6: Build an interface demonstrating concept of multiple inheritance.
// Subject: SBL - OOP Java
// Student: Anish Vyapari | Roll No: 25CA1012 | PRN: DY25ENGU0AIM012

// First Interface: Printable
interface Printable {
    // Public static final constant by default
    String STANDARD_PAGE_SIZE = "A4 (210 x 297 mm)";

    // Abstract method contract
    void printDocument();
    void configurePageLayout(String orientation);
}

// Second Interface: SecureCloudSharable
interface SecureCloudSharable {
    // Public static final constant
    String ENCRYPTION_STANDARD = "AES-256 GCM";

    // Abstract method contracts
    void uploadToCloud(String endpointUrl);
    boolean verifyChecksum(String expectedHash);
}

// Third Interface: Auditable (Demonstrating interface inheritance)
interface Auditable {
    void logAccessHistory(String user);
}

// Concrete Class implementing MULTIPLE INTERFACES (Printable, SecureCloudSharable, Auditable)
// This achieves Multiple Inheritance of Type and Behavior in Java
class TechnicalReport implements Printable, SecureCloudSharable, Auditable {
    private String reportId;
    private String title;
    private String author;
    private int pageCount;
    private String documentHash;

    public TechnicalReport(String reportId, String title, String author, int pageCount, String
documentHash) {
        this.reportId = reportId;
        this.title = title;
        this.author = author;
        this.pageCount = pageCount;
        this.documentHash = documentHash;
    }

    // Implementing methods from interface Printable
    @Override
    public void printDocument() {
        System.out.println(" [Print Service]: Printing Report → ID: " + reportId);
        System.out.println(" Title      : \"" + title + "\"");
        System.out.println(" Author    : " + author);
        System.out.println(" Total Pages : " + pageCount + " pages on " + STANDARD_PAGE_SIZE);
        System.out.println(" Status     : Output dispatched to high-resolution spooler
successfully.");
    }

    @Override
    public void configurePageLayout(String orientation) {
        System.out.println(" [Print Service]: Page Layout configured to " + orientation + "
```

```

orientation.");
    }

    // Implementing methods from interface SecureCloudSharable
    @Override
    public void uploadToCloud(String endpointUrl) {
        System.out.println(" [Cloud Service]: Initializing encrypted payload transfer...");
        System.out.println(" Destination : " + endpointUrl);
        System.out.println(" Cipher Mode : " + ENCRYPTION_STANDARD);
        System.out.println(" Transfer      : Secure handshake established. File transmitted
successfully.");
    }

    @Override
    public boolean verifyChecksum(String expectedHash) {
        boolean match = this.documentHash.equalsIgnoreCase(expectedHash);
        System.out.println(" [Security]: Verifying SHA-256 integrity hash → Match Result: " +
match);
        return match;
    }

    // Implementing method from interface Auditable
    @Override
    public void logAccessHistory(String user) {
        System.out.println(" [Audit Trail]: Access granted to user [" + user + "] at timestamp:
2026-09-28 10:15:30 IST");
    }
}

public class Exp6Interfaces {
    public static void main(String[] args) {
        System.out.println("=== RAIT Nerul - SBL OOP Java Lab: Experiment 6 ===");
        System.out.println("Candidate: Anish Vyapari | Roll No: 25CA1012");
        System.out.println("Topic: Multiple Inheritance Concept using Java Interfaces\n");

        System.out.println("=====");
        System.out.println("CREATING INSTANCE OF CLASS IMPLEMENTING MULTIPLE INTERFACES");
        System.out.println("=====");

        TechnicalReport report = new TechnicalReport(
            "RAIT-SBL-2026-001",
            "Advanced OOP Architecture and Design Patterns in Java",
            "Anish Vyapari",
            18,
            "e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855"
        );

        System.out.println("\n--- Invoking Printable Interface Capabilities ---");
        report.configurePageLayout("Portrait");
        report.printDocument();

        System.out.println("\n--- Invoking SecureCloudSharable Interface Capabilities ---");
        report.uploadToCloud("https://cloud.rait.ac.in/vault/sbl-java/records");
        report.verifyChecksum("e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855");

        System.out.println("\n--- Invoking Auditable Interface Capabilities ---");
        report.logAccessHistory("Anish_Vyapari_25CA1012");

        System.out.println("\n--- Demonstrating Polymorphic Interface References (Upcasting) ---
");

        // Printable reference holding the TechnicalReport object
        Printable pRef = report;
        System.out.println("Printable Reference → Standard Format: " +
Printable.STANDARD_PAGE_SIZE);
    }
}

```

```
pRef.printDocument();

// SecureCloudSharable reference holding the same object
SecureCloudSharable sRef = report;
System.out.println("SecureCloudSharable Reference → Encryption: " +
SecureCloudSharable.ENCRIPTION_STANDARD);
sRef.uploadToCloud("https://backup-node.rait.ac.in/replica");

System.out.println("=====");
}
}
```

■ CONCLUSION

Successfully designed and implemented multiple interfaces in Java, demonstrating how Java achieves multiple inheritance of type and behavioral contracts without the ambiguities of the Diamond Problem. Confirmed interface polymorphism, contract enforcement, and loose coupling.

EXPERIMENT 7

Write a program to demonstrate exception handling using try, catch, throw, throws and finally.

■ AIM

To design and execute a Java program demonstrating robust, fault-tolerant exception handling using all five exception keywords: try, catch, throw, throws, and finally.

■ OBJECTIVE

- Understand the Java Exception Hierarchy rooted at `java.lang.Throwable`.
- Differentiate between Checked (compile-time) and Unchecked (runtime) exceptions.
- Implement try and catch blocks to intercept runtime anomalies and maintain application stability.
- Use throw to raise exceptions programmatically and throws to declare exception propagation.
- Implement finally blocks to guarantee cleanup and resource integrity regardless of execution outcome.

■ SOFTWARE REQUIRED

Java Development Kit (JDK 21 LTS), Visual Studio Code / Eclipse IDE, Windows Terminal.

■ THEORY

1. Architectural Philosophy of Exception Handling in Java

An **exception** is an abnormal event or fault condition that arises during program execution, disrupting the normal flow of instructions. In procedural languages like C, error handling relies on checking error return codes, which intermixes business logic with error detection and can easily be ignored by developers. Java implements a structured, object-oriented exception handling model that separates error handling from core application workflows.

The Java Exception Hierarchy is rooted at `java.lang.Throwable`, which branches into two primary subclasses:

1. **Error**: Represents catastrophic, unrecoverable system failures originating within the JVM environment (e.g., `OutOfMemoryError`, `StackOverflowError`). Applications should not attempt to catch Errors.
2. **Exception**: Represents recoverable conditions that a well-written application can anticipate and handle gracefully. It divides into:
 - **Checked Exceptions**: Subclasses of `Exception` excluding `RuntimeException`. Enforced by the compiler at compile time; methods must either catch them or declare them via `throws` (e.g., `IOException`, `SQLException`).
 - **Unchecked Exceptions (Runtime Exceptions)**: Subclasses of `RuntimeException`. Represent programming flaws such as logic bugs, null pointer dereferences, or illegal arithmetic operations (e.g., `ArithmeticException`, `IllegalArgumentException`).

2. The Five Exception Handling Keywords

Java's exception handling mechanism is governed by five keywords:

- **try**: Encloses a block of code monitored for exceptional conditions. If an exception occurs within the `try` block, execution halts immediately and control transfers to an appropriate catch handler.

- **catch**: Defines an exception handler block designed to intercept and process specific exception types. Multiple **catch** blocks can be chained to handle different exception classes. Catch blocks must be ordered from the most specific subclass to the most general superclass; placing a superclass handler before a subclass handler causes an unreachable code compile error.
- **throw**: Used to explicitly generate and throw an exception object at a specific point in code:

```
if (amount > balance) {
    throw new ArithmeticException("Insufficient Balance!");
}
```

- **throws**: Appended to a method signature to declare that the method may propagate specified exceptions up the call stack to its caller:

```
public void withdraw(double amt) throws IllegalArgumentException, ArithmeticException { ...
```

- **finally**: Defines a cleanup block that is **guaranteed to execute**, regardless of whether an exception was thrown, caught, or unhandled. It executes even if the **try** or **catch** block contains a **return** statement. The **finally** block is essential for releasing critical system resources (database connections, file descriptors, network sockets) to prevent resource leaks.

3. Call Stack Unwinding and Exception Propagation

When an exception is thrown, the JVM searches the current method's call stack for a matching **catch** block. If none is found, the current method terminates abruptly, its stack frame is popped, and the search continues in the calling method. This process—known as **call stack unwinding**—continues until a matching catch block is located or the exception reaches the main thread, resulting in default exception handler logging and thread termination.

■ ALGORITHM

1. Start the program.
2. Define class BankAccount with private attributes: accountNumber, accountHolder, and balance.
3. Implement constructor BankAccount(accountNumber, accountHolder, balance) to initialize account.
4. Implement method withdraw(double amount) declaring throws IllegalArgumentException, ArithmeticException:
5. a. If amount <= 0, throw new IllegalArgumentException('Withdrawal amount must be strictly greater than zero!').
6. b. If amount > balance, throw new ArithmeticException('Insufficient Balance!').
7. c. Deduct amount from balance and display success confirmation.
8. Implement helper method executeTransaction(BankAccount account, double amount) declaring throws to demonstrate method-level exception propagation.
9. In main(), instantiate BankAccount with initial balance Rs. 25000.0.
10. Define an array of test transaction amounts: {5000.0, -1200.0, 35000.0, 10000.0}.
11. Iterate through transaction requests; for each cycle:
12. a. Enter try block and invoke executeTransaction().
13. b. Provide catch (IllegalArgumentException ex) to handle negative/zero amounts.
14. c. Provide catch (ArithmeticException ex) to handle insufficient balance conditions.
15. d. Provide fallback catch (Exception ex) to handle unexpected anomalies.
16. e. Implement finally block: commit transaction audit log, flush state, and display post-cycle balance.

17. Display completion message verifying continuous execution without abnormal application termination.

18. Stop.

■ PROGRAM

```
// Experiment 7: Write a program to demonstrate exception handling using try, catch, throw, throws
// and finally.
// Subject: SBL - OOP Java
// Student: Anish Vyapari | Roll No: 25CA1012 | PRN: DY25ENGU0AIM012

class BankAccount {
    private String accountNumber;
    private String accountHolder;
    private double balance;

    public BankAccount(String accountNumber, String accountHolder, double balance) {
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = balance;
    }

    // Demonstrating 'throws' keyword in method declaration
    // Informs callers that this method may throw IllegalArgumentException or ArithmeticException
    public void withdraw(double amount) throws IllegalArgumentException, ArithmeticException {
        System.out.println(" [Attempting Withdrawal]: Request for Rs. " + amount + " on Account "
+ accountNumber);

        // 1. Demonstrating 'throw' keyword for non-positive withdrawal amount
        if (amount ≤ 0) {
            throw new IllegalArgumentException("Withdrawal amount must be strictly greater than
zero! (Supplied: " + amount + ")");
        }

        // 2. Demonstrating 'throw' keyword for business logic boundary condition
        if (amount > balance) {
            throw new ArithmeticException("Insufficient Balance! Requested: Rs. " + amount + ",
Available: Rs. " + balance);
        }

        balance -= amount;
        System.out.printf(" Withdrawal Successful! Dispensed: Rs. %.2f | Remaining Balance: Rs.
%.2f\n", amount, balance);
    }

    public double getBalance() {
        return balance;
    }
}

public class Exp7ExceptionHandling {

    // Helper method demonstrating method-level exception propagation using 'throws'
    public static void executeTransaction(BankAccount account, double amount) throws
IllegalArgumentException, ArithmeticException {
        account.withdraw(amount);
    }

    public static void main(String[] args) {
        System.out.println("=== RAIT Nerul - SBL OOP Java Lab: Experiment 7 ===");
        System.out.println("Candidate: Anish Vyapari | Roll No: 25CA1012");
        System.out.println("Topic: Exception Handling using try, catch, throw, throws, and
finally\n");
    }
}
```

```

        BankAccount myAccount = new BankAccount("AC-1012-DYP", "Anish Vyapari", 25000.0);
        System.out.println("Initial Account State: Holder: Anish Vyapari | Balance: Rs. " +
myAccount.getBalance());

        double[] transactionRequests = {5000.0, -1200.0, 35000.0, 10000.0};

        for (int i = 0; i < transactionRequests.length; i++) {
            double reqAmount = transactionRequests[i];
            System.out.println("\n=====");
            System.out.println("TRANSACTION CYCLE #" + (i + 1));
            System.out.println("=====");

            // 1. 'try' block enclosing code that may throw exceptions
            try {
                System.out.println(">>> Entering [try] block...");
                // Calling method declared with 'throws'
                executeTransaction(myAccount, reqAmount);
                System.out.println(">>> Transaction processed without runtime exceptions.");
            }
            // 2. 'catch' block 1: Specific handling for IllegalArgumentException
            catch (IllegalArgumentException ex) {
                System.out.println(">>> [catch - IllegalArgumentException Caught]: " +
ex.getMessage());
            }
            // 3. 'catch' block 2: Specific handling for ArithmeticException
            catch (ArithmeticException ex) {
                System.out.println(">>> [catch - ArithmeticException Caught]: " +
ex.getMessage());
            }
            // 4. 'catch' block 3: Generic fallback Exception handler
            catch (Exception ex) {
                System.out.println(">>> [catch - Generic Exception Caught]: " + ex.toString());
            }
            // 5. 'finally' block: Always executes regardless of whether an exception occurred or
was caught
            finally {
                System.out.println(">>> [finally] Block Executed: Audit log committed. DB
connection flushed. Session state intact.");
                System.out.printf(">>> End of Cycle Balance: Rs. %.2f\n", myAccount.getBalance());
            }
        }

        System.out.println("\n=====");
        System.out.println("APPLICATION EXECUTION CONTINUES NORMALLY TO CONCLUSION");
        System.out.println("=====");
    }
}

```

■ CONCLUSION

Successfully demonstrated robust, structured exception handling in Java using try, catch, throw, throws, and finally. Verified exception propagation across method boundaries, isolated illegal parameters and overdraft violations into specific catch handlers, and confirmed guaranteed execution of finally cleanup blocks.

EXPERIMENT 8

Design and implement user defined exception handling.

■ AIM

To design and implement custom (user-defined) checked exception classes in Java, enforce application-specific domain business rules, throw custom exceptions, and catch/handle them gracefully.

■ OBJECTIVE

- Understand the architectural rationale for domain-specific user-defined exceptions in enterprise systems.
- Design custom checked exception classes by extending `java.lang.Exception`.
- Incorporate custom error codes, diagnostic fields, and override `toString()` and `getMessage()`.
- Enforce business rule invariants by programmatically throwing and handling custom exceptions.

■ SOFTWARE REQUIRED

Java Development Kit (JDK 21 LTS), Visual Studio Code / Eclipse IDE, Windows Terminal.

■ THEORY

1. Rationale for User-Defined (Custom) Exceptions

While the Java Standard Library provides a comprehensive collection of built-in exceptions (e.g., `NullPointerException`, `IndexOutOfBoundsException`, `IllegalArgumentException`), these standard types represent generic programmatic flaws. Enterprise applications, however, govern complex problem domains—such as financial banking rules, healthcare compliance, or electoral validation—with unique business constraints.

Standard exceptions cannot convey domain-specific semantics. For instance, throwing a generic `IllegalArgumentException` when an underage citizen attempts voter registration obscures the true business violation. **User-Defined Exceptions** (custom exceptions) solve this by introducing explicit domain concepts into the type system, yielding self-documenting code, distinct error codes, and rich diagnostic data.

2. Designing Custom Exception Classes

To design a custom exception, a Java class must extend an existing member of the `Throwable` hierarchy:

- **Checked Custom Exception (Extending `java.lang.Exception`):** The Java compiler enforces that callers must either catch the exception using `try-catch` or declare it in their method signature using `throws`. This is the recommended approach for recoverable business logic violations where the caller must be forced to handle the anomaly.
- **Unchecked Custom Exception (Extending `java.lang.RuntimeException`):** The compiler does not mandate explicit handling. Used primarily for internal logical bugs or unrecoverable programming faults.

3. Anatomy of a Robust Custom Exception

A production-grade custom exception should encompass:

1. **Call to Superclass Constructor:** Invoking `super(message)` ensures that the descriptive error message is registered in the base `Throwable` class and accessible via `getMessage()`.

2. **Diagnostic State Attributes:** Storing contextual metadata relevant to the failure (e.g., the illegal age supplied, legal threshold required, conflicting unique identifiers).
3. **Structured toString() Representation:** Overriding `toString()` to output standardized enterprise error codes (e.g., `[ErrorCode: VOTE-ERR-001]`) alongside contextual parameters for automated log analysis and monitoring.

```
public class InvalidVoterAgeException extends Exception {
    private int suppliedAge;
    private int legalAgeRequired;

    public InvalidVoterAgeException(String msg, int age, int required) {
        super(msg);
        this.suppliedAge = age;
        this.legalAgeRequired = required;
    }
    // Diagnostic accessors and toString()
}
```

4. Business Invariant Enforcement and Defensive Programming

Custom exceptions empower domain services to enforce business invariants defensively. Rather than returning error flags or silent nulls, domain methods validate inputs and immediately `throw` domain exceptions when constraints are breached. This guarantees that invalid entity states cannot persist into storage registries.

■ ALGORITHM

1. Start the program.
2. Define custom checked exception `InvalidVoterAgeException` extending `Exception` with fields `suppliedAge` and `legalAgeRequired`, constructor, and overridden `toString()` with error code `VOTE-ERR-001`.
3. Define custom checked exception `DuplicateVoterRegistrationException` extending `Exception` with field `voterId`, constructor, and overridden `toString()` with error code `VOTE-ERR-002`.
4. Define model class `VoterRecord` with fields: `voterId`, `fullName`, `age`, `constituency`, and `displayCard()` method.
5. Define service class `ElectoralRegistrationPortal` managing an internal array of `VoterRecord` objects.
6. Implement `registerNewElector(voterId, fullName, age, constituency)` declaring `throws InvalidVoterAgeException, DuplicateVoterRegistrationException`:
7. a. Check age invariant: if `age < 18`, throw new `InvalidVoterAgeException('Eligibility Violation...', age, 18)`.
8. b. Check duplicate invariant: scan existing registry; if any existing record matches `voterId`, throw new `DuplicateVoterRegistrationException('Integrity Violation...', voterId)`.
9. c. If validation passes, instantiate `VoterRecord`, append to registry array, and display confirmation card.
10. In `main()`, initialize `ElectoralRegistrationPortal`.
11. Define test cases: candidate 1 (valid age 21), candidate 2 (underage age 16), candidate 3 (valid age 24), candidate 4 (duplicate submission with ID of candidate 1).
12. Iterate through submissions; wrap each in try block.
13. Catch `InvalidVoterAgeException`: extract supplied age and minimum legal age diagnostics.
14. Catch `DuplicateVoterRegistrationException`: extract conflicting voter ID diagnostic.
15. Catch general `Exception` as fallback.

16. Verify all enrollment cycles complete safely without unhandled crashes.

17. Stop.

■ PROGRAM

```
// Experiment 8: Design and implement user defined exception handling.
// Subject: SBL - OOP Java
// Student: Anish Vyapari | Roll No: 25CA1012 | PRN: DY25ENGU0AIM012

// 1. Custom Checked Exception: Underage Voter
class InvalidVoterAgeException extends Exception {
    private int suppliedAge;
    private int legalAgeRequired;

    public InvalidVoterAgeException(String message, int suppliedAge, int legalAgeRequired) {
        super(message);
        this.suppliedAge = suppliedAge;
        this.legalAgeRequired = legalAgeRequired;
    }

    public int getSuppliedAge() {
        return suppliedAge;
    }

    public int getLegalAgeRequired() {
        return legalAgeRequired;
    }

    @Override
    public String toString() {
        return "InvalidVoterAgeException [ErrorCode: VOTE-ERR-001]: " + getMessage() +
            " (Age Given: " + suppliedAge + ", Minimum Legal Age: " + legalAgeRequired + ")";
    }
}

// 2. Custom Checked Exception: Duplicate Registration
class DuplicateVoterRegistrationException extends Exception {
    private String voterId;

    public DuplicateVoterRegistrationException(String message, String voterId) {
        super(message);
        this.voterId = voterId;
    }

    public String getVoterId() {
        return voterId;
    }

    @Override
    public String toString() {
        return "DuplicateVoterRegistrationException [ErrorCode: VOTE-ERR-002]: " + getMessage() +
            " (Conflicting Voter ID: " + voterId + ")";
    }
}

// Model Class: Voter
class VoterRecord {
    private String voterId;
    private String fullName;
    private int age;
    private String constituency;
}
```

```

    public VoterRecord(String voterId, String fullName, int age, String constituency) {
        this.voterId = voterId;
        this.fullName = fullName;
        this.age = age;
        this.constituency = constituency;
    }

    public String getVoterId() {
        return voterId;
    }

    public void displayCard() {
        System.out.println(" [VOTER ID CARD ISSUED]");
        System.out.println(" Card Number : " + voterId);
        System.out.println(" Elector Name : " + fullName);
        System.out.println(" Age : " + age + " years");
        System.out.println(" Constituency : " + constituency);
        System.out.println(" Status : Active & Enrolled in Electoral Roll");
    }
}

// Service / Controller Class
class ElectoralRegistrationPortal {
    private VoterRecord[] registeredVoters;
    private int count;

    public ElectoralRegistrationPortal(int capacity) {
        registeredVoters = new VoterRecord[capacity];
        count = 0;
    }

    // Method enforcing business rules and throwing custom user-defined exceptions
    public void registerNewElector(String voterId, String fullName, int age, String constituency)
        throws InvalidVoterAgeException, DuplicateVoterRegistrationException {

        System.out.println("\n[Portal Processing Registration]: Candidate: " + fullName + ", Age: " + age + ", ID: " + voterId);

        // Rule 1: Age verification
        if (age < 18) {
            throw new InvalidVoterAgeException(
                "Eligibility Violation: Citizen must be at least 18 years old to exercise constitutional franchise.",
                age,
                18
            );
        }

        // Rule 2: Duplicate ID check
        for (int i = 0; i < count; i++) {
            if (registeredVoters[i].getVoterId().equalsIgnoreCase(voterId)) {
                throw new DuplicateVoterRegistrationException(
                    "Integrity Violation: A voter record with this electoral identification number already exists in the system registry.",
                    voterId
                );
            }
        }

        // Add valid record
        VoterRecord newVoter = new VoterRecord(voterId, fullName, age, constituency);
        registeredVoters[count++] = newVoter;
        System.out.println("Registration Approved!");
        newVoter.displayCard();
    }
}

```

```

    }
}

public class Exp8CustomExceptions {
    public static void main(String[] args) {
        System.out.println("=== RAIT Nerul - SBL OOP Java Lab: Experiment 8 ===");
        System.out.println("Candidate: Anish Vyapari | Roll No: 25CA1012");
        System.out.println("Topic: Design and Implementation of User Defined Exceptions\n");

        ElectoralRegistrationPortal portal = new ElectoralRegistrationPortal(10);

        // Test Cases designed to trigger normal registration and both custom exceptions
        Object[][] testApplications = {
            {"MH-NERUL-101", "Anish Vyapari", 21, "Belapur / Nerul Division"},
            {"MH-NERUL-102", "Pooja Deshmukh", 16, "Thane Urban Division"},
            {"MH-NERUL-103", "Sameer Patil", 24, "Navi Mumbai Central"},
            {"MH-NERUL-101", "Anish Vyapari (Duplicate Re-entry)", 21, "Belapur / Nerul Division"}
        };

        for (int i = 0; i < testApplications.length; i++) {
            String id = (String) testApplications[i][0];
            String name = (String) testApplications[i][1];
            int age = (Integer) testApplications[i][2];
            String constName = (String) testApplications[i][3];

            System.out.println("=====");
            System.out.println("ENROLLMENT SUBMISSION REQUEST #" + (i + 1));
            System.out.println("=====");

            try {
                portal.registerNewElector(id, name, age, constName);
            } catch (InvalidVoterAgeException ex) {
                System.out.println("\n[CAUGHT USER-DEFINED EXCEPTION]");
                System.out.println("Message : " + ex.getMessage());
                System.out.println("Diagnostics: Given Age = " + ex.getSuppliedAge() + ", Required = " + ex.getLegalAgeRequired());
                System.out.println("toString : " + ex.toString());
            } catch (DuplicateVoterRegistrationException ex) {
                System.out.println("\n[CAUGHT USER-DEFINED EXCEPTION]");
                System.out.println("Message : " + ex.getMessage());
                System.out.println("Diagnostics: Conflicting Voter ID = " + ex.getVoterId());
                System.out.println("toString : " + ex.toString());
            } catch (Exception ex) {
                System.out.println("Unexpected Exception Caught: " + ex.getMessage());
            }
        }

        System.out.println("\n=====");
        System.out.println("ALL ENROLLMENT CYCLES HANDLED SAFELY VIA CUSTOM EXCEPTIONS");
        System.out.println("=====");
    }
}

```

■ CONCLUSION

Successfully designed, implemented, and verified user-defined checked exceptions in Java. Demonstrated how custom exception classes extending `java.lang.Exception` enforce domain business rules, enrich diagnostic logs with error codes, and facilitate graceful error recovery in enterprise systems.

EXPERIMENT 9

Construct the component and container class to construct/design GUI.

■ AIM

To construct and design a graphical user interface (GUI) in Java using Swing container classes (JFrame, JPanel) and component classes (JLabel, JTextField, JButton, JComboBox, JRadioButton, JCheckBox, JTextArea) with event handling.

■ OBJECTIVE

- Understand the architectural distinction between AWT heavyweight components and Java Swing lightweight components.
- Structure GUI layouts using Top-Level Containers (JFrame), Intermediate Containers (JPanel), and Layout Managers.
- Implement Atomic Component controls: JLabel, JTextField, JRadioButton, ButtonGroup, JComboBox, JCheckBox, JButton, and JTextArea.
- Implement the Delegation Event Model using ActionListener on the Event Dispatch Thread (EDT).

■ SOFTWARE REQUIRED

Java Development Kit (JDK 21 LTS), Java Foundation Classes (Swing / AWT), Windows Terminal.

■ THEORY

1. Java Graphical User Interface (GUI) Architecture: AWT vs Swing

Java provides two primary toolkits for desktop graphical interface construction:

- **Abstract Window Toolkit (AWT):** Java's initial GUI framework. AWT components are *heavyweight* because they delegate their rendering and event processing directly to the underlying operating system's native GUI peers (Windows, X11, Cocoa). Consequently, AWT interfaces look and behave differently on each OS, consume high system resources, and are constrained to the lowest common denominator of platform capabilities.
- **Java Swing:** Part of the Java Foundation Classes (JFC), built on top of AWT. Swing components are *lightweight* because they are rendered entirely in pure Java using the Java 2D API without native OS peers (with the exception of top-level windows). Swing provides a *Pluggable Look and Feel (PLAF)*, allowing identical visual appearance and behavior across all operating platforms, alongside advanced UI components such as trees, tables, and tabbed panes.

2. Container and Component Hierarchy

The visual elements in Java Swing inherit from a structured class hierarchy rooted at `java.awt.Component` → `java.awt.Container` → `javax.swing.JComponent`:

1. **Top-Level Containers:** Heavyweight windows that provide the operating system frame, title bar, minimize/maximize controls, and close buttons. Every Swing GUI requires at least one top-level container, typically `JFrame` or `JDialog`.
2. **Intermediate Containers:** Lightweight container panels such as `JPanel` and `JScrollPane` that group, organize, and format visual sub-sections within top-level containers.

3. Atomic (Leaf) Components: Interactive or display controls that reside inside containers to interact with users:

- `JLabel` : Displays passive text or graphic icons.
- `TextField` : Single-line editable text input field.
- `Button` : Push button triggering actionable events.
- `JRadioButton` & `ButtonGroup` : Mutually exclusive selection controls.
- `JComboBox` : Drop-down combination list box.
- `JCheckBox` : Multi-selection toggle controls.
- `TextArea` : Multi-line text display and editing area.

3. Layout Management Architecture

In Java, components are never positioned with rigid hardcoded pixel coordinates, which would break across varying monitor resolutions and display scaling factors. Instead, positioning is delegated to **Layout Managers**:

- `BorderLayout` : Divides container into five regions: `NORTH` , `SOUTH` , `EAST` , `WEST` , and `CENTER` .
- `GridLayout` : Arranges components in a rigid two-dimensional rectangular grid of equal-sized cells.
- `FlowLayout` : Places components in a directional flow line, wrapping to the next line when boundaries are reached.
- `GridBagLayout` : The most powerful and flexible layout manager, aligning components in a dynamic grid using `GridBagConstraints` to control cell spanning, weights, and insets.

4. The Delegation Event Model and Event Dispatch Thread (EDT)

Swing operates under the **Delegation Event Model**:

- **Event Source:** A user interface component (e.g., `JButton`) that detects user interactions.
- **Event Object:** An encapsulated event instance (e.g., `ActionEvent`) generated by the source when interacted with.
- **Event Listener:** An interface (e.g., `ActionListener`) registered with the source via `addActionListener(this)` .

When the button is clicked, the Event Source delegates the `ActionEvent` to the registered listener by invoking `actionPerformed(ActionEvent e)` on the **Event Dispatch Thread (EDT)**, ensuring thread-safe GUI updates.

■ ALGORITHM

1. Start the program.
2. Create class `Exp9SwingGUI` extending `JFrame` and implementing `ActionListener`.
3. In constructor, configure frame properties: title, size (780×680), close operation (`EXIT_ON_CLOSE`), and set main layout to `BorderLayout(12, 12)`.
4. Construct North Container: `JPanel` with `BoxLayout(Y_AXIS)`, academic blue background, containing header `JLabel` components for RAIT Nerul and SBL Java GUI Workbench.
5. Construct Center Container: `JPanel` with `GridLayout(1, 2)` dividing window into Form Input Panel (left) and Ledger Display Panel (right).
6. Populate Left Form Panel using `GridBagLayout` with components:
7. a. Add `JLabel` and `TextField` for Full Name, Roll Number, and PRN.
8. b. Add `JRadioButton` for Male and Female; group them in `ButtonGroup` for mutual exclusivity.

9. c. Add JComboBox for Academic Department and Semester selections.
10. d. Add JCheckBox controls for Course Modules (SBL OOP Java, Data Structures, Discrete Math).
11. e. Add action buttons: Submit (Green), Reset (Amber), Exit (Red) inside a flow button subpanel.
12. Populate Right Display Panel: add JTextArea wrapped in a JScrollPane to display real-time confirmation records.
13. Construct South Container: status bar JPanel displaying active EDT status.
14. Register ActionListener on Submit, Reset, and Exit buttons.
15. In actionPerformed(ActionEvent e):
 16. a. If Submit clicked: extract text, radio selection, combo items, and selected checkboxes; format official confirmation ledger; display in JTextArea.
 17. b. If Reset clicked: clear all text fields, reset combo selections, and restore default checkboxes.
 18. c. If Exit clicked: terminate application.
19. In main(), schedule GUI instantiation on the Event Dispatch Thread via SwingUtilities.invokeLater().
20. Capture programmatic GUI snapshot to exp9_gui_window.png and display window.
21. Stop.

■ PROGRAM

```
// Experiment 9: Construct the component and container class to construct/design GUI.
// Subject: SBL - OOP Java
// Student: Anish Vyapari | Roll No: 25CA1012 | PRN: DY25ENGU0A1M012

import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.border.TitledBorder;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.image.BufferedImage;
import java.io.File;
import javax.imageio.ImageIO;

// Container Class: StudentEnrollmentGUI extends JFrame (Top-level container)
public class Exp9SwingGUI extends JFrame implements ActionListener {

    // Component Declarations
    private JTextField txtName, txtRoll, txtPrn, txtEmail;
    private JRadioButton rbMale, rbFemale;
    private ButtonGroup genderGroup;
    private JComboBox<String> cbBranch, cbSemester;
    private JCheckBox chkOopJava, chkDataStructures, chkDisSys;
    private JButton btnSubmit, btnReset, btnExit;
    private JTextArea txtOutput;

    public Exp9SwingGUI() {
        // 1. Frame Configuration (Top-level Window Container)
        super("RAIT Nerul - SBL OOP Java Lab: Student Enrollment System");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(780, 680);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout(12, 12));
        getContentPane().setBackground(new Color(245, 247, 250));

        // 2. Header Panel (North Container)
        JPanel pnlHeader = new JPanel();
        pnlHeader.setBackground(new Color(26, 54, 93));
```

```

        pnlHeader.setLayout(new BorderLayout(pnlHeader, BorderLayout.Y_AXIS));
        pnlHeader.setBorder(new EmptyBorder(12, 15, 12, 15));

        JLabel lblTitle = new JLabel("RAMRAO ADIK INSTITUTE OF TECHNOLOGY, NERUL");
        lblTitle.setFont(new Font("Segoe UI", Font.BOLD, 17));
        lblTitle.setForeground(Color.WHITE);
        lblTitle.setAlignmentX(Component.CENTER_ALIGNMENT);

        JLabel lblSub = new JLabel("Department of Computer Engineering | SBL - OOP Java GUI
Workbench");
        lblSub.setFont(new Font("Segoe UI", Font.PLAIN, 12));
        lblSub.setForeground(new Color(210, 225, 245));
        lblSub.setAlignmentX(Component.CENTER_ALIGNMENT);

        pnlHeader.add(lblTitle);
        pnlHeader.add(Box.createVerticalStrut(4));
        pnlHeader.add(lblSub);
        add(pnlHeader, BorderLayout.NORTH);

        // 3. Central Working Panel (Container with GridBagLayout)
        JPanel pnlCenter = new JPanel(new GridLayout(1, 2, 12, 12));
        pnlCenter.setBackground(new Color(245, 247, 250));
        pnlCenter.setBorder(new EmptyBorder(0, 15, 0, 15));

        // Left Form Panel: Component Container
        JPanel pnlForm = new JPanel(new GridBagLayout());
        pnlForm.setBackground(Color.WHITE);
        pnlForm.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(new Color(203, 213, 225), 1),
            BorderFactory.createTitledBorder(
                BorderFactory.createEmptyBorder(5, 5, 5, 5),
                "Enrollment Input Controls (Components)",
                TitledBorder.LEFT,
                TitledBorder.TOP,
                new Font("Segoe UI", Font.BOLD, 13),
                new Color(30, 41, 59)
            )
        ));

        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(5, 8, 5, 8);
        gbc.fill = GridBagConstraints.HORIZONTAL;

        // Row 0: Full Name
        gbc.gridx = 0; gbc.gridy = 0; gbc.weightx = 0.3;
        pnlForm.add(new JLabel("Full Name:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.7;
        txtName = new JTextField("Anish Vyapari", 15);
        pnlForm.add(txtName, gbc);

        // Row 1: Roll Number
        gbc.gridx = 0; gbc.gridy = 1; gbc.weightx = 0.3;
        pnlForm.add(new JLabel("Roll Number:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.7;
        txtRoll = new JTextField("25CA1012", 15);
        pnlForm.add(txtRoll, gbc);

        // Row 2: PRN
        gbc.gridx = 0; gbc.gridy = 2; gbc.weightx = 0.3;
        pnlForm.add(new JLabel("PRN:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.7;
        txtPrn = new JTextField("DY25ENGU0AIM012", 15);
        pnlForm.add(txtPrn, gbc);

```

```

// Row 3: Gender (Radio Buttons in ButtonGroup)
gbc.gridx = 0; gbc.gridy = 3; gbc.weightx = 0.3;
pnlForm.add(new JLabel("Gender:"), gbc);
gbc.gridx = 1; gbc.weightx = 0.7;
JPanel pnlGender = new JPanel(new FlowLayout(FlowLayout.LEFT, 0, 0));
pnlGender.setBackground(Color.WHITE);
rbMale = new JRadioButton("Male", true);
rbFemale = new JRadioButton("Female");
rbMale.setBackground(Color.WHITE);
rbFemale.setBackground(Color.WHITE);
genderGroup = new ButtonGroup();
genderGroup.add(rbMale);
genderGroup.add(rbFemale);
pnlGender.add(rbMale);
pnlGender.add(Box.createHorizontalStrut(12));
pnlGender.add(rbFemale);
pnlForm.add(pnlGender, gbc);

// Row 4: Academic Program (JComboBox)
gbc.gridx = 0; gbc.gridy = 4; gbc.weightx = 0.3;
pnlForm.add(new JLabel("Department:"), gbc);
gbc.gridx = 1; gbc.weightx = 0.7;
String[] branches = {"Artificial Intelligence & Data Science", "Computer Engineering",
"Information Technology", "Electronics & Telecomm."};
cbBranch = new JComboBox<>(branches);
pnlForm.add(cbBranch, gbc);

// Row 5: Semester (JComboBox)
gbc.gridx = 0; gbc.gridy = 5; gbc.weightx = 0.3;
pnlForm.add(new JLabel("Semester:"), gbc);
gbc.gridx = 1; gbc.weightx = 0.7;
String[] semesters = {"Semester 3 (SE)", "Semester 4 (SE)", "Semester 5 (TE)", "Semester 6
(TE)"};
cbSemester = new JComboBox<>(semesters);
pnlForm.add(cbSemester, gbc);

// Row 6: Elective Courses (JCheckBox)
gbc.gridx = 0; gbc.gridy = 6; gbc.weightx = 0.3;
pnlForm.add(new JLabel("Course Modules:"), gbc);
gbc.gridx = 1; gbc.weightx = 0.7;
JPanel pnlCourses = new JPanel(new GridLayout(3, 1, 2, 2));
pnlCourses.setBackground(Color.WHITE);
chkOopJava = new JCheckBox("SBL - OOP Java (Core)", true);
chkDataStructures = new JCheckBox("Data Structures Lab", true);
chkDisSys = new JCheckBox("Discrete Mathematical Structures", false);
chkOopJava.setBackground(Color.WHITE);
chkDataStructures.setBackground(Color.WHITE);
chkDisSys.setBackground(Color.WHITE);
pnlCourses.add(chkOopJava);
pnlCourses.add(chkDataStructures);
pnlCourses.add(chkDisSys);
pnlForm.add(pnlCourses, gbc);

// Row 7: Action Buttons (FlowLayout Container)
gbc.gridx = 0; gbc.gridy = 7; gbc.gridwidth = 2;
JPanel pnlButtons = new JPanel(new FlowLayout(FlowLayout.CENTER, 8, 8));
pnlButtons.setBackground(Color.WHITE);

btnSubmit = new JButton("Register Record");
btnReset = new JButton("Clear Fields");
btnExit = new JButton("Exit");

btnSubmit.setBackground(new Color(21, 128, 61));
btnSubmit.setForeground(Color.WHITE);

```

```

        btnSubmit.setFocusPainted(false);

        btnReset.setBackground(new Color(217, 119, 6));
        btnReset.setForeground(Color.WHITE);
        btnReset.setFocusPainted(false);

        btnExit.setBackground(new Color(185, 28, 28));
        btnExit.setForeground(Color.WHITE);
        btnExit.setFocusPainted(false);

        btnSubmit.addActionListener(this);
        btnReset.addActionListener(this);
        btnExit.addActionListener(this);

        pnlButtons.add(btnSubmit);
        pnlButtons.add(btnReset);
        pnlButtons.add(btnExit);
        pnlForm.add(pnlButtons, gbc);

        // Right Output Panel: JTextArea within JScrollPane
        JPanel pnlRight = new JPanel(new BorderLayout(5, 5));
        pnlRight.setBackground(Color.WHITE);
        pnlRight.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(new Color(203, 213, 225), 1),
            BorderFactory.createTitledBorder(
                BorderFactory.createEmptyBorder(5, 5, 5, 5),
                "System Confirmation Ledger (Display Component)",
                TitledBorder.LEFT,
                TitledBorder.TOP,
                new Font("Segoe UI", Font.BOLD, 13),
                new Color(30, 41, 59)
            )
        ));

        txtOutput = new JTextArea();
        txtOutput.setEditable(false);
        txtOutput.setFont(new Font("Consolas", Font.PLAIN, 12));
        txtOutput.setBackground(new Color(248, 250, 252));
        txtOutput.setText(">>> SYSTEM READY: Fill details and click 'Register Record'.\n");
        JScrollPane scrollPane = new JScrollPane(txtOutput);
        pnlRight.add(scrollPane, BorderLayout.CENTER);

        pnlCenter.add(pnlForm);
        pnlCenter.add(pnlRight);
        add(pnlCenter, BorderLayout.CENTER);

        // 4. Status Bar (South Container)
        JPanel pnlStatus = new JPanel(new BorderLayout());
        pnlStatus.setBackground(new Color(226, 232, 240));
        pnlStatus.setBorder(new EmptyBorder(5, 15, 5, 15));
        JLabel lblStatus = new JLabel("GUI Status: Event Dispatch Thread Active | AWT/Swing Components Initialized");
        lblStatus.setFont(new Font("Segoe UI", Font.ITALIC, 11));
        pnlStatus.add(lblStatus, BorderLayout.WEST);
        add(pnlStatus, BorderLayout.SOUTH);
    }

    // Event Handling implementation for ActionListener
    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == btnSubmit) {
            String name = txtName.getText().trim();
            String roll = txtRoll.getText().trim();
            String prn = txtPrn.getText().trim();

```

```

        String gender = rbMale.isSelected() ? "Male" : "Female";
        String branch = (String) cbBranch.getSelectedItem();
        String sem = (String) cbSemester.getSelectedItem();

        StringBuilder sbCourses = new StringBuilder();
        if (chkOopJava.isSelected()) sbCourses.append(" [x] SBL - OOP Java (Core)\n");
        if (chkDataStructures.isSelected()) sbCourses.append(" [x] Data Structures Lab\n");
        if (chkDisSys.isSelected()) sbCourses.append(" [x] Discrete Mathematical
Structures\n");

        StringBuilder log = new StringBuilder();
        log.append("=====\n");
        log.append("          OFFICIAL ENROLLMENT CONFIRMATION          \n");
        log.append("=====\n");
        log.append("Candidate Name   : ").append(name).append("\n");
        log.append("Roll Number      : ").append(roll).append("\n");
        log.append("PRN Identifier    : ").append(prn).append("\n");
        log.append("Gender            : ").append(gender).append("\n");
        log.append("Branch / Stream   : ").append(branch).append("\n");
        log.append("Academic Term     : ").append(sem).append("\n");
        log.append("Enrolled Modules : \n").append(sbCourses);
        log.append("Timestamp         : 2026-09-28 11:20:45 IST\n");
        log.append("Registration Status: SUCCESS - RECORD STORED IN LMS\n");
        log.append("=====\n");

        txtOutput.setText(log.toString());
        System.out.println("[GUI Event]: Form submitted and processed successfully for " +
name);
    } else if (e.getSource() == btnReset) {
        txtName.setText("");
        txtRoll.setText("");
        txtPrn.setText("");
        rbMale.setSelected(true);
        cbBranch.setSelectedIndex(0);
        cbSemester.setSelectedIndex(0);
        chkOopJava.setSelected(true);
        chkDataStructures.setSelected(false);
        chkDisSys.setSelected(false);
        txtOutput.setText(">>> Form cleared by user.\n");
        System.out.println("[GUI Event]: Form reset triggered.");
    } else if (e.getSource() == btnExit) {
        System.out.println("[GUI Event]: Exit requested.");
        System.exit(0);
    }
}

// Method to trigger automatic programmatic submission and snapshot export
public void generateSnapshotAndExit(String outputPath) {
    // Trigger submit action programmatically
    btnSubmit.doClick();

    // Allow components to layout and paint
    setSize(780, 680);
    doLayout();
    validate();

    // Create BufferedImage offscreen
    BufferedImage img = new BufferedImage(getWidth(), getHeight(),
BufferedImage.TYPE_INT_RGB);
    Graphics2D g2d = img.createGraphics();
    g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
    g2d.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING,
RenderingHints.VALUE_TEXT_ANTIALIASING);
    paint(g2d);
}

```

```

        g2d.dispose();

        try {
            File outFile = new File(outputPath);
            ImageIO.write(img, "png", outFile);
            System.out.println("GUI Snapshot successfully saved to: " +
outFile.getAbsolutePath());
            System.exit(0);
        } catch (Exception ex) {
            System.err.println("Error saving snapshot: " + ex.getMessage());
        }
    }

    public static void main(String[] args) {
        System.out.println("≡≡≡ RAIT Nerul - SBL OOP Java Lab: Experiment 9 ≡≡≡");
        System.out.println("Candidate: Anish Vyapari | Roll No: 25CA1012");
        System.out.println("Topic: Component and Container Classes for GUI Design in Java
Swing\n");

        SwingUtilities.invokeLater(() → {
            Exp9SwingGUI gui = new Exp9SwingGUI();
            gui.setVisible(true);

            // Export high-resolution snapshot for lab report
            gui.generateSnapshotAndExit("exp9_gui_window.png");
        });
    }
}

```

■ CONCLUSION

Successfully designed and constructed an interactive desktop GUI application using Java Swing container classes (JFrame, JPanel, JScrollPane) and component classes (JLabel, JTextField, JButton, JRadioButton, JComboBox, JCheckBox, JTextArea). Verified layout management and event handling via the Delegation Event Model.

EXPERIMENT 10

Build a dynamic feature using Java based script.

■ AIM

To build a dynamic feature in a Java application using a Java-based scripting and expression evaluation engine, demonstrating runtime variable binding, dynamic rule parsing, and execution without code recompilation.

■ OBJECTIVE

- Analyze the trade-offs between static compile-time typing and runtime dynamic extensibility in enterprise Java architectures.
- Understand the architecture of Java's Scripting API (JSR-223 javax.script framework).
- Implement an object-oriented Dynamic Scripting Engine in pure Java that evaluates conditional and mathematical script expressions at runtime.
- Bind Java domain entities to the dynamic scripting execution environment and execute user-configurable business rules.

■ SOFTWARE REQUIRED

Java Development Kit (JDK 21 LTS), Visual Studio Code / Eclipse IDE, Windows Terminal.

■ THEORY

1. Static Compilation vs Dynamic Extensibility in Enterprise Java

Java is fundamentally a statically typed, compiled programming language. Source code is verified by `javac` and converted into immutable bytecode ahead-of-time. While this ensures high performance, memory safety, and early defect detection, modern enterprise systems—such as financial trading platforms, insurance underwriting engines, dynamic pricing algorithms, and ERP workflow engines—require high business adaptability.

Business rules, commission structures, and eligibility criteria change frequently. Hardcoding these rules into static Java classes requires recompiling, testing, and redeploying the entire software package for every minor policy revision. To overcome this limitation, Java applications incorporate **dynamic scripting engines** that interpret and evaluate dynamic rules and scripts at runtime while executing against compiled Java domain objects.

2. Architecture of the Java Scripting Framework (JSR-223)

Java standardizes scripting integration through **JSR-223: Scripting for the Java Platform** (packaged in `javax.script`). The architecture consists of:

- **ScriptEngineManager**: A service provider discovery mechanism that scans the application classpath for registered script engines (such as JavaScript, Groovy, or Python).
- **ScriptEngine**: The core interpreter interface responsible for evaluating scripts and executing dynamic expressions.
- **Bindings**: A key-value map interface representing the execution context. Variables from the host Java application are injected into the script environment, allowing scripts to read and manipulate live Java objects.

3. Building a Pure Java Dynamic Rule and Scripting Engine

In standard enterprise environments where external third-party script engines may be restricted, dynamic features are implemented in pure Java using an object-oriented **Dynamic Scripting Engine**. This engine implements a domain-specific scripting language (DSL) supporting:

1. **Dynamic Variable Binding:** An environment map (`Map<String, Object>`) that binds Java domain entity attributes (e.g., `basicSalary`, `quarterlySales`, `clientRetentionRate`) into the script symbol table at runtime.
2. **Conditional Script Syntax:** Parsing constructs such as:

```
IF <condition> THEN <true_expression> ELSE <false_expression>
```

3. **Relational & Arithmetic Evaluators:** Tokenizing conditional operators (`>=`, `<=`, `=`, `!=`, `>`, `<`) and arithmetic operators (`+`, `-`, `*`, `/`) to resolve expressions dynamically without bytecode compilation.

4. Practical and Industrial Applications

Java-based dynamic scripting engines power real-world business rule management systems (BRMS) such as Apache Drools and Camunda BPM, credit scoring algorithms in fintech, customizable e-commerce discounting, and automated academic grading workflows.

■ ALGORITHM

1. Start the program.
2. Define domain entity class `SalesExecutive` with fields: `id`, `name`, `basicSalary`, `quarterlySales`, `clientRetentionRate`, `performanceScore`.
3. Define class `JavaDynamicScriptEngine` with an internal `Map<String, Object>` `environmentBindings`.
4. Implement `bindValue(String key, Object value)` to inject domain entity attributes into the script execution context.
5. Implement `clearBindings()` to reset the script context between evaluations.
6. Implement `evaluateScript(String script)`:
 7. a. Check if script starts with 'IF '; extract `conditionPart`, `truePart`, and `falsePart`.
 8. b. In `evaluateBooleanCondition()`: parse relational operators (`>=`, `<=`, `==`, `!=`, `>`, `<`); resolve variable names to bound numbers; evaluate condition truth value.
 9. c. If condition is true, execute `evaluateArithmeticExpression(truePart)`; otherwise, execute `evaluateArithmeticExpression(falsePart)`.
 10. d. In `evaluateArithmeticExpression()`: substitute bound variable names with current numeric values; parse and compute arithmetic operations adhering to operator precedence.
11. In `main()`, initialize `JavaDynamicScriptEngine`.
12. Instantiate team of `SalesExecutive` domain objects with diverse performance profiles.
13. Define runtime dynamic scripts for quarterly performance incentives and client retention bonuses.
14. Iterate through executives; bind properties to script engine; evaluate dynamic incentive and retention scripts; calculate total remuneration; display execution traces and results.
15. Stop.

■ PROGRAM

```
// Experiment 10: Build a dynamic feature using Java based script.
// Subject: SBL - OOP Java
// Student: Anish Vyapari | Roll No: 25CA1012 | PRN: DY25ENGU0AIM012
```

```

import java.util.*;

// Domain Entity: SalesExecutive
class SalesExecutive {
    private String id;
    private String name;
    private double basicSalary;
    private double quarterlySales;
    private int clientRetentionRate; // percentage (0-100)
    private double performanceScore; // 1.0 to 5.0

    public SalesExecutive(String id, String name, double basicSalary, double quarterlySales, int
clientRetentionRate, double performanceScore) {
        this.id = id;
        this.name = name;
        this.basicSalary = basicSalary;
        this.quarterlySales = quarterlySales;
        this.clientRetentionRate = clientRetentionRate;
        this.performanceScore = performanceScore;
    }

    public String getId() { return id; }
    public String getName() { return name; }
    public double getBasicSalary() { return basicSalary; }
    public double getQuarterlySales() { return quarterlySales; }
    public int getClientRetentionRate() { return clientRetentionRate; }
    public double getPerformanceScore() { return performanceScore; }
}

// Dynamic Scripting Engine implemented in Java OOP
// Provides dynamic feature execution: parses and evaluates dynamic scripting rules on Java
objects at runtime
class JavaDynamicScriptEngine {
    private Map<String, Object> environmentBindings;

    public JavaDynamicScriptEngine() {
        this.environmentBindings = new HashMap<>();
    }

    // Bind Java domain object variables into the script execution context
    public void bindVariable(String key, Object value) {
        environmentBindings.put(key, value);
    }

    // Clears context bindings
    public void clearBindings() {
        environmentBindings.clear();
    }

    // Evaluates dynamic mathematical and logical scripting expressions at runtime
    // Syntax supported: "IF <condition> THEN <expr1> ELSE <expr2>" or direct mathematical
expression
    public double evaluateScript(String script) throws Exception {
        script = script.trim();

        // 1. Check for Conditional Dynamic Script: IF ... THEN ... ELSE ...
        if (script.toUpperCase().startsWith("IF ")) {
            int thenIdx = script.toUpperCase().indexOf(" THEN ");
            int elseIdx = script.toUpperCase().indexOf(" ELSE ");

            if (thenIdx == -1 || elseIdx == -1 || elseIdx < thenIdx) {
                throw new IllegalArgumentException("Syntax Error in Dynamic Script: Expected IF
<condition> THEN <true_expr> ELSE <false_expr>");
            }
        }
    }
}

```

```

    }

    String conditionPart = script.substring(3, thenIdx).trim();
    String truePart = script.substring(thenIdx + 6, elseIdx).trim();
    String falsePart = script.substring(elseIdx + 6).trim();

    boolean conditionResult = evaluateBooleanCondition(conditionPart);
    System.out.println("    [Script Trace] Condition: (" + conditionPart + ") → Evaluated
To: " + conditionResult);

    if (conditionResult) {
        System.out.println("    [Script Trace] Branch Taken: THEN → " + truePart);
        return evaluateArithmeticExpression(truePart);
    } else {
        System.out.println("    [Script Trace] Branch Taken: ELSE → " + falsePart);
        return evaluateArithmeticExpression(falsePart);
    }
} else {
    // Direct mathematical script
    return evaluateArithmeticExpression(script);
}
}

// Evaluates condition: e.g. "quarterlySales ≥ 500000" or "performanceScore > 4.0"
private boolean evaluateBooleanCondition(String condition) {
    String[] operators = {"≥", "≤", "=", "≠", ">", "<"};
    for (String op : operators) {
        int opIdx = condition.indexOf(op);
        if (opIdx ≠ -1) {
            String leftToken = condition.substring(0, opIdx).trim();
            String rightToken = condition.substring(opIdx + op.length()).trim();

            double leftVal = resolveOperand(leftToken);
            double rightVal = resolveOperand(rightToken);

            switch (op) {
                case "≥": return leftVal ≥ rightVal;
                case "≤": return leftVal ≤ rightVal;
                case "=": return Math.abs(leftVal - rightVal) < 0.0001;
                case "≠": return Math.abs(leftVal - rightVal) ≥ 0.0001;
                case ">": return leftVal > rightVal;
                case "<": return leftVal < rightVal;
            }
        }
    }
    throw new IllegalArgumentException("Unsupported conditional operator in script: " +
condition);
}

// Evaluates dynamic arithmetic expressions with operators +, -, *, /
private double evaluateArithmeticExpression(String expression) {
    // Substitute all bound variables
    String processed = expression;
    for (Map.Entry<String, Object> entry : environmentBindings.entrySet()) {
        if (entry.getValue() instanceof Number) {
            processed = processed.replaceAll("\\b" + entry.getKey() + "\\b",
entry.getValue().toString());
        }
    }

    // Tokenize and evaluate expression using standard arithmetic parser
    return parseSimpleExpression(processed);
}

```

```

// Resolves token to either variable value from environment or numeric literal
private double resolveOperand(String token) {
    if (environmentBindings.containsKey(token)) {
        Object val = environmentBindings.get(token);
        if (val instanceof Number) {
            return ((Number) val).doubleValue();
        }
    }
    return Double.parseDouble(token);
}

// Evaluates basic math expressions: handles +, -, *
private double parseSimpleExpression(String expr) {
    expr = expr.replaceAll("\\s+", "");
    // Handle addition and subtraction (lower precedence)
    List<String> terms = new ArrayList<>();
    List<Character> termOps = new ArrayList<>();

    int start = 0;
    for (int i = 0; i < expr.length(); i++) {
        char c = expr.charAt(i);
        if ((c == '+' || c == '-') && i > 0 && expr.charAt(i - 1) != '*' && expr.charAt(i - 1)
            != '/') {
            terms.add(expr.substring(start, i));
            termOps.add(c);
            start = i + 1;
        }
    }
    terms.add(expr.substring(start));

    double result = parseTerm(terms.get(0));
    for (int i = 0; i < termOps.size(); i++) {
        char op = termOps.get(i);
        double nextVal = parseTerm(terms.get(i + 1));
        if (op == '+') result += nextVal;
        else if (op == '-') result -= nextVal;
    }
    return result;
}

private double parseTerm(String term) {
    // Handle multiplication and division
    String[] factors = term.split("\\*");
    double product = 1.0;
    for (String factor : factors) {
        if (factor.contains("/")) {
            String[] divParts = factor.split("/");
            double divResult = resolveOperand(divParts[0]);
            for (int d = 1; d < divParts.length; d++) {
                divResult /= resolveOperand(divParts[d]);
            }
            product *= divResult;
        } else {
            product *= resolveOperand(factor);
        }
    }
    return product;
}

public class Exp10DynamicScript {
    public static void main(String[] args) {
        System.out.println("=== RAIT Nerul - SBL OOP Java Lab: Experiment 10 ===");
        System.out.println("Candidate: Anish Vyapari | Roll No: 25CA1012");
    }
}

```

```

        System.out.println("Topic: Dynamic Feature Implementation using Java-Based Scripting
Engine\n");

        // 1. Initialize Dynamic Scripting Engine
        JavaDynamicScriptEngine scriptEngine = new JavaDynamicScriptEngine();

        // 2. Define Candidate Domain Objects
        List<SalesExecutive> team = Arrays.asList(
            new SalesExecutive("SE-101", "Anish Vyapari", 75000.0, 850000.0, 94, 4.8),
            new SalesExecutive("SE-102", "Priya Nair", 60000.0, 420000.0, 78, 3.7),
            new SalesExecutive("SE-103", "Vikram Rathod", 65000.0, 620000.0, 88, 4.2)
        );

        // 3. User-Defined Dynamic Script Rules (Can be modified at runtime without
recompilation!)
        String dynamicIncentiveScript =
            "IF quarterlySales ≥ 500000 THEN basicSalary * 0.25 + 15000 ELSE basicSalary * 0.10 +
2000";

        String dynamicRetentionBonusScript =
            "IF clientRetentionRate ≥ 90 THEN 12000 ELSE 4000";

        System.out.println("=====");
        System.out.println("DYNAMIC SCRIPT RULES LOADED AT RUNTIME:");
        System.out.println("=====");
        System.out.println("Rule 1 (Incentive Script):");
        System.out.println("  \"\" + dynamicIncentiveScript + "\"\"");
        System.out.println("Rule 2 (Retention Bonus Script):");
        System.out.println("  \"\" + dynamicRetentionBonusScript + "\"\"");

        System.out.println("=====");
        System.out.println("EVALUATING DYNAMIC RULES ACROSS DOMAIN OBJECTS");
        System.out.println("=====");

        for (SalesExecutive exec : team) {
            System.out.println("\n>>> Processing Executive: " + exec.getName() + " (ID: " +
exec.getId() + ")");
            System.out.println("    Profile: Base Salary = Rs. " + exec.getBasicSalary() +
                " | Quarterly Sales = Rs. " + exec.getQuarterlySales() +
                " | Retention = " + exec.getClientRetentionRate() + "%" +
                " | Rating = " + exec.getPerformanceScore());

            // Bind Executive's runtime properties to Script Engine Context
            scriptEngine.bindVariable("basicSalary", exec.getBasicSalary());
            scriptEngine.bindVariable("quarterlySales", exec.getQuarterlySales());
            scriptEngine.bindVariable("clientRetentionRate", exec.getClientRetentionRate());
            scriptEngine.bindVariable("performanceScore", exec.getPerformanceScore());

            try {
                // Execute Dynamic Incentive Script
                System.out.println("    Executing Dynamic Incentive Calculation:");
                double incentive = scriptEngine.evaluateScript(dynamicIncentiveScript);

                // Execute Dynamic Retention Bonus Script
                System.out.println("    Executing Dynamic Retention Bonus Calculation:");
                double retentionBonus = scriptEngine.evaluateScript(dynamicRetentionBonusScript);

                double totalPayout = exec.getBasicSalary() + incentive + retentionBonus;

                System.out.printf("    [Calculated Results via Dynamic Script]:\n");
                System.out.printf("      Base Salary      : Rs. %10.2f\n", exec.getBasicSalary());
                System.out.printf("      Dynamic Incentive: Rs. %10.2f\n", incentive);
                System.out.printf("      Retention Bonus  : Rs. %10.2f\n", retentionBonus);
                System.out.printf("      Total Payout     : Rs. %10.2f\n", totalPayout);
            }
        }
    }
}

```

```

        } catch (Exception ex) {
            System.out.println(" [Script Error]: " + ex.getMessage());
        } finally {
            scriptEngine.clearBindings();
        }
    }

    System.out.println("\n=====");
    System.out.println("DYNAMIC SCRIPT EXECUTION COMPLETED SUCCESSFULLY");
    System.out.println("=====");
}
}

```

■ CONCLUSION

Successfully designed and implemented a dynamic feature evaluation engine in pure OOP Java. Demonstrated runtime variable binding, dynamic script parsing, and conditional rule execution across live domain objects, proving how Java applications can achieve dynamic business adaptability without code recompilation.

EXPERIMENT 11

Capstone Project: Smart Banking Transaction and Account Portfolio Management System (SB-TMS)

■ AIM

To design and implement a comprehensive, secure, object-oriented Banking Transaction and Account Portfolio Management System that integrates core OOP principles including encapsulation, inheritance hierarchies, abstract classes, multiple interfaces, method and constructor overloading, 1D/2D array ledger processing, custom exception handling, and transaction auditing.

■ OBJECTIVE

- Architect an integrated enterprise domain model featuring an abstract base class `Account` and specialized derived classes `SavingsAccount` and `CurrentAccount`.
- Implement multiple interface contracts (`TransactionProcessor` and `Auditable`) to enforce clean service boundaries and loose coupling.
- Enforce business banking invariants via custom checked exceptions: `InsufficientFundsException` and `OverdraftExceededException`.
- Maintain an immutable transaction ledger using 2D array structures to generate official passbook statements.
- Demonstrate compile-time polymorphism (overloaded deposits/transfers) and runtime polymorphism (dynamic dispatch on interest accrual and withdrawals).

■ SOFTWARE REQUIRED

Java Development Kit (JDK 21 LTS), Visual Studio Code / Eclipse IDE, Windows Terminal.

■ THEORY

1. Architectural Blueprint of the Capstone Project

The **Smart Banking Transaction and Account Portfolio Management System (SB-TMS)** serves as the culminating capstone project for the SBL – OOP Java laboratory. It synthesizes every major object-oriented software engineering concept mastered across Experiments 1 through 10 into a cohesive, industrial-grade banking ledger application.

The system architecture is structured across four decoupled layers:

1. **Contract Layer:** Formal interfaces (`TransactionProcessor` , `Auditable`) defining strict operational contracts for deposits, withdrawals, transfers, and ledger auditing.
2. **Domain Model Layer:** An abstract base class `Account` encapsulating common account attributes, constructor overloading, and 2D array ledger logging, extended by concrete accounts (`SavingsAccount` , `CurrentAccount`).
3. **Resilience & Exception Layer:** Custom domain exceptions (`InsufficientFundsException` , `OverdraftExceededException`) that intercept illegal banking actions.
4. **Service & Orchestration Layer:** Coordinated transaction workflows including inter-account wire transfers, quarterly polymorphic interest/fee batch servicing, and passbook statement generation.

2. Mapping to Foundational Object-Oriented Principles

OOP Principle	Concrete Implementation in SB-TMS
Encapsulation	Sensitive account balances, ledger arrays, and account identifiers are declared <code>protected</code> / <code>private</code> , shielded from direct external modification and accessible strictly via validated business methods.
Inheritance	<code>SavingsAccount</code> and <code>CurrentAccount</code> extend abstract base class <code>Account</code> , inheriting core state, transaction logging routines, and constructor chaining via <code>super(...)</code> .
Polymorphism	Compile-time polymorphism via overloaded <code>deposit()</code> methods. Runtime polymorphism via dynamic method dispatch on overridden <code>withdraw()</code> and <code>applyPeriodicInterest()</code> .
Abstraction	Abstract class <code>Account</code> and interfaces <code>TransactionProcessor</code> and <code>Auditable</code> define essential contracts while concealing implementation specifics.

3. Multi-Dimensional Array Ledger Architecture

Rather than relying on ephemeral screen output, the base `Account` maintains an internal two-dimensional String array `transactionLedger[50][4]` acting as an in-memory passbook ledger. Each transaction event is recorded with four structured columns: `[Transaction ID, Type, Amount, Net Balance]`. The system traverses this 2D array structure to format and print official ASCII passbook statements.

4. Robust Fault Tolerance via Custom Domain Exceptions

The system guarantees data consistency by rejecting invalid transactions through custom checked exceptions: `InsufficientFundsException` (triggered when a savings withdrawal breaches the Rs. 2000 minimum reserve) and `OverdraftExceededException` (triggered when a commercial account exhausts its approved credit facility).

■ ALGORITHM

1. Start the capstone program.
2. Define custom checked exceptions `InsufficientFundsException` and `OverdraftExceededException` with contextual diagnostic fields and formatted error strings.
3. Define interface `TransactionProcessor` specifying `deposit(amount)`, overloaded `deposit(amount, note)`, `withdraw(amount)`, and `transfer(recipient, amount)`.
4. Define interface `Auditable` specifying `printStatement()` and `generateAuditSummary()`.
5. Define abstract base class `Account` implementing `TransactionProcessor` and `Auditable`:
6. a. Declare protected attributes: `accountNumber`, `holderName`, `balance`, `transactionLedger[50][4]`, `txCount`.
7. b. Implement overloaded constructors: `Account(accNo, holder)` and `Account(accNo, holder, initialDeposit)`.
8. c. Implement `recordTransaction(type, amount, postBalance)` to log records into the 2D array ledger.
9. d. Implement overloaded `deposit(amount)` and `deposit(amount, note)`.
10. e. Implement `transfer(recipient, amount)`: coordinate `withdraw()` from source account and `deposit()` to destination account.
11. f. Declare abstract method `applyPeriodicInterest()`.
12. g. Implement `printStatement()`: iterate through 2D array ledger to display official passbook table.
13. Define derived class `SavingsAccount` extending `Account`:

14. a. Declare `annualInterestRate` and `MINIMUM_OPERATING_BALANCE = 2000.0`.
15. b. Override `withdraw()`: if `balance - amount < minimum balance`, throw `InsufficientFundsException`.
16. c. Override `applyPeriodicInterest()`: compute quarterly interest credit and log to ledger.
17. Define derived class `CurrentAccount` extending `Account`:
18. a. Declare `overdraftLimit` and `usedOverdraft`.
19. b. Override `withdraw()`: allow withdrawal using balance; if deficit, utilize overdraft up to limit; throw `OverdraftExceededException` if exceeded.
20. c. Override `applyPeriodicInterest()`: levy 3% fee on active overdraft usage.
21. In `main()`, provision portfolio: `SavingsAccount` (Anish Vyapari), `CurrentAccount` (Vyapari Tech Solutions LLP), external `SavingsAccount` (Kavita Rao).
22. Execute standard deposits, withdrawals, and inter-account wire transfer.
23. Execute boundary exception tests: trigger Savings minimum balance violation and Current account overdraft exhaustion.
24. Execute batch end-of-quarter servicing across `Account[]` array using runtime polymorphic dynamic dispatch.
25. Generate official 2D array passbook statements for all portfolio accounts.
26. Stop.

■ PROGRAM

```
// Experiment 11: Capstone Project
// Project Title: Smart Banking Transaction and Account Portfolio Management System (SB-TMS)
// Subject: SBL - OOP Java
// Student: Anish Vyapari | Roll No: 25CA1012 | PRN: DY25ENGU0AIM012

import java.util.*;

// --- CUSTOM DOMAIN EXCEPTIONS ---

class InsufficientFundsException extends Exception {
    private double balance;
    private double withdrawalAmount;

    public InsufficientFundsException(String message, double balance, double withdrawalAmount) {
        super(message);
        this.balance = balance;
        this.withdrawalAmount = withdrawalAmount;
    }

    @Override
    public String toString() {
        return "InsufficientFundsException [ERR-BANK-001]: " + getMessage() +
            " (Available: Rs. " + balance + ", Attempted: Rs. " + withdrawalAmount + ")";
    }
}

class OverdraftExceededException extends Exception {
    private double currentOverdraft;
    private double maxOverdraftLimit;

    public OverdraftExceededException(String message, double currentOverdraft, double
maxOverdraftLimit) {
        super(message);
        this.currentOverdraft = currentOverdraft;
        this.maxOverdraftLimit = maxOverdraftLimit;
    }
}
```

```

    }

    @Override
    public String toString() {
        return "OverdraftExceededException [ERR-BANK-002]: " + getMessage() +
            " (Overdraft Requested: Rs. " + currentOverdraft + ", Approved Limit: Rs. " +
maxOverdraftLimit + ")";
    }
}

// --- INTERFACES FOR MULTIPLE CONTRACT SPECIFICATION ---

interface TransactionProcessor {
    void deposit(double amount);
    void deposit(double amount, String referenceNote); // Overloaded method
    void withdraw(double amount) throws InsufficientFundsException, OverdraftExceededException;
    void transfer(Account recipient, double amount) throws InsufficientFundsException,
OverdraftExceededException;
}

interface Auditable {
    void printStatement();
    void generateAuditSummary();
}

// --- ABSTRACT BASE CLASS ---

abstract class Account implements TransactionProcessor, Auditable {
    protected String accountNumber;
    protected String holderName;
    protected double balance;
    protected String[][] transactionLedger; // 2D Array for Transaction History [Row: Records,
Col: [ID, Type, Amount, Balance]]
    protected int txCount;

    // Base Constructor Overloading
    public Account(String accountNumber, String holderName) {
        this(accountNumber, holderName, 0.0);
    }

    public Account(String accountNumber, String holderName, double initialDeposit) {
        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.balance = initialDeposit;
        this.transactionLedger = new String[50][4];
        this.txCount = 0;
        recordTransaction("OPEN", initialDeposit, balance);
    }

    // Helper to log transaction into 2D Array
    protected void recordTransaction(String type, double amount, double postBalance) {
        if (txCount < transactionLedger.length) {
            transactionLedger[txCount][0] = "TXN-" + String.format("%04d", (txCount + 1));
            transactionLedger[txCount][1] = type;
            transactionLedger[txCount][2] = String.format("%.2f", amount);
            transactionLedger[txCount][3] = String.format("%.2f", postBalance);
            txCount++;
        }
    }

    // Method Overloading: deposit(amount)
    @Override
    public void deposit(double amount) {
        deposit(amount, "Standard Counter Deposit");
    }
}

```

```

    }

    // Method Overloading: deposit(amount, note)
    @Override
    public void deposit(double amount, String referenceNote) {
        if (amount > 0) {
            balance += amount;
            recordTransaction("DEP", amount, balance);
            System.out.printf(" [Deposit Confirmed]: Rs. %.2f deposited into [%s] (%s) | Balance:
Rs. %.2f\n",
                                amount, accountNumber, referenceNote, balance);
        } else {
            System.out.println(" [Deposit Error]: Invalid deposit amount.");
        }
    }

    // Transfer implementation demonstrating Polymorphic interactions
    @Override
    public void transfer(Account recipient, double amount) throws InsufficientFundsException,
OverdraftExceededException {
        System.out.println("\n [Initiating Inter-Account Wire Transfer]:");
        System.out.printf(" From: %s (%s) → To: %s (%s) | Amount: Rs. %.2f\n",
            this.accountNumber, this.holderName, recipient.accountNumber,
recipient.holderName, amount);

        this.withdraw(amount); // May throw domain exception
        recipient.deposit(amount, "Wire Inflow from " + this.accountNumber);
        System.out.println(" [Wire Transfer Completed Successfully!]");
    }

    // Abstract method to be overridden by derived accounts (Runtime Polymorphism)
    public abstract void applyPeriodicInterest();

    @Override
    public void printStatement() {
        System.out.println("\n-----
-----");
        System.out.printf("OFFICIAL PASSBOOK STATEMENT | Account: %s | Holder: %s\n",
accountNumber, holderName);
        System.out.println("-----
-----");
        System.out.printf("%-10s | %-8s | %-15s | %-15s\n", "TXN ID", "TYPE", "AMOUNT (Rs.)",
"BALANCE (Rs.)");
        System.out.println("-----
-----");
        for (int i = 0; i < txCount; i++) {
            System.out.printf("%-10s | %-8s | %-15s | %-15s\n",
                transactionLedger[i][0],
                transactionLedger[i][1],
                transactionLedger[i][2],
                transactionLedger[i][3]);
        }
        System.out.println("-----
-----");
        System.out.printf("Closing Net Balance: Rs. %.2f\n", balance);
        System.out.println("-----
-----");
    }

    public String getAccountNumber() { return accountNumber; }
    public String getHolderName() { return holderName; }
    public double getBalance() { return balance; }
}

```

```
// --- SUBCLASS 1: SAVINGS ACCOUNT (Inheritance & Super keyword) ---

class SavingsAccount extends Account {
    private double annualInterestRate; // e.g. 4.5%
    private static final double MINIMUM_OPERATING_BALANCE = 2000.0;

    public SavingsAccount(String accountNumber, String holderName, double initialDeposit, double
interestRate) {
        super(accountNumber, holderName, initialDeposit);
        this.annualInterestRate = interestRate;
    }

    @Override
    public void withdraw(double amount) throws InsufficientFundsException {
        if (amount ≤ 0) {
            throw new IllegalArgumentException("Withdrawal request must be positive.");
        }
        if ((balance - amount) < MINIMUM_OPERATING_BALANCE) {
            throw new InsufficientFundsException(
                "Savings Minimum Balance Rule Violated (Required Threshold: Rs. " +
MINIMUM_OPERATING_BALANCE + ")",
                balance, amount
            );
        }
        balance -= amount;
        recordTransaction("WDL", amount, balance);
        System.out.printf(" [Savings Withdrawal]: Rs. %.2f dispensed from [%s] | Balance: Rs.
%.2f\n", amount, accountNumber, balance);
    }

    @Override
    public void applyPeriodicInterest() {
        double quarterlyInterest = (balance * (annualInterestRate / 100.0)) / 4.0;
        balance += quarterlyInterest;
        recordTransaction("INT", quarterlyInterest, balance);
        System.out.printf(" [Interest Credit]: Added quarterly interest Rs. %.2f (@%.1f%% p.a.) |
New Balance: Rs. %.2f\n",
            quarterlyInterest, annualInterestRate, balance);
    }

    @Override
    public void generateAuditSummary() {
        System.out.println(" [Audit Metric]: Savings Account " + accountNumber + " maintains
compliance with minimum reserve.");
    }
}

// --- SUBCLASS 2: CURRENT ACCOUNT (Inheritance with Overdraft Support) ---

class CurrentAccount extends Account {
    private double overdraftLimit;
    private double usedOverdraft;

    public CurrentAccount(String accountNumber, String holderName, double initialDeposit, double
overdraftLimit) {
        super(accountNumber, holderName, initialDeposit);
        this.overdraftLimit = overdraftLimit;
        this.usedOverdraft = 0.0;
    }

    @Override
    public void withdraw(double amount) throws InsufficientFundsException,
OverdraftExceededException {
        if (amount ≤ 0) {
```

```

        throw new IllegalArgumentException("Withdrawal request must be positive.");
    }

    if (amount ≤ balance) {
        balance -= amount;
        recordTransaction("WDL", amount, balance);
        System.out.printf(" [Current Withdrawal]: Rs. %.2f debited from [%s] | Balance: Rs.
%.2f\n", amount, accountNumber, balance);
    } else {
        // Need overdraft facility
        double deficit = amount - balance;
        if ((usedOverdraft + deficit) > overdraftLimit) {
            throw new OverdraftExceededException(
                "Overdraft Limit Exhausted! Deficit required: Rs. " + deficit,
                (usedOverdraft + deficit), overdraftLimit
            );
        }
        usedOverdraft += deficit;
        balance = 0.0;
        recordTransaction("WDL-OD", amount, balance);
        System.out.printf(" [Current Overdraft Withdrawal]: Rs. %.2f debited. Current
Overdraft Utilized: Rs. %.2f / Rs. %.2f\n",
            amount, usedOverdraft, overdraftLimit);
    }
}

@Override
public void applyPeriodicInterest() {
    if (usedOverdraft > 0) {
        double overdraftInterestFee = usedOverdraft * 0.03; // 3% fee on active overdraft
        usedOverdraft += overdraftInterestFee;
        System.out.printf(" [Overdraft Levy]: Added overdraft utilization fee of Rs. %.2f |
Outstanding OD: Rs. %.2f\n",
            overdraftInterestFee, usedOverdraft);
    } else {
        System.out.println(" [Current Account]: Commercial accounts earn zero interest. No
overdraft charges incurred.");
    }
}

@Override
public void generateAuditSummary() {
    System.out.printf(" [Audit Metric]: Current Account %s | Active Overdraft: Rs. %.2f / Rs.
%.2f\n",
        accountNumber, usedOverdraft, overdraftLimit);
}
}

// --- CAPSTONE APPLICATION ORCHESTRATOR ---

public class Exp11CapstoneProject {
    public static void main(String[] args) {

System.out.println("=====
=");
        System.out.println("    RAMRAO ADIK INSTITUTE OF TECHNOLOGY, NERUL - DEPARTMENT OF COMPUTER
ENGG.    ");
        System.out.println("                SBL - OBJECT ORIENTED PROGRAMMING IN JAVA LABORATORY
");
        System.out.println("                CAPSTONE PROJECT: SMART BANKING & TRANSACTION SYSTEM (SB-
TMS)    ");
        System.out.println("    Candidate: Anish Vyapari | Roll No: 25CA1012 | PRN: DY25ENGU00AIM012
");
    }
}

```

```

System.out.println("=====
=\n");

    // 1. Account Portfolio Initialization (Constructor Overloading & Polymorphism)
    System.out.println(">>> 1. INITIALIZING BANKING PORTFOLIO ACCOUNTS:");
    SavingsAccount accSavings = new SavingsAccount("SB-1012-IN", "Anish Vyapari", 50000.0,
4.25);
    CurrentAccount accCorporate = new CurrentAccount("CA-9080-CORP", "Vyapari Tech Solutions
LLP", 100000.0, 50000.0);
    SavingsAccount accThirdParty = new SavingsAccount("SB-3045-EXT", "Kavita Rao", 15000.0,
4.0);

    System.out.println(" → Accounts Provisioned Successfully in Core Banking Registry.\n");

    // 2. Executing Standard Transactions
    System.out.println(">>> 2. PERFORMING CORE TRANSACTIONS (DEPOSITS & WITHDRAWALS:");
    accSavings.deposit(12500.0, "Semester Academic Scholarship Credit");
    accCorporate.deposit(45000.0, "Enterprise Client Invoice Settlement");

    try {
        accSavings.withdraw(8000.0);
        accCorporate.withdraw(120000.0); // Uses balance + 20,000 overdraft
    } catch (Exception e) {
        System.out.println(" [Exception]: " + e.getMessage());
    }

    // 3. Demonstrating Inter-Account Polymorphic Transfer
    System.out.println("\n>>> 3. DEMONSTRATING INTER-ACCOUNT WIRE TRANSFER:");
    try {
        accSavings.transfer(accThirdParty, 15000.0);
    } catch (Exception e) {
        System.out.println(" [Transfer Fault]: " + e.getMessage());
    }

    // 4. Demonstrating Exception Handling & Domain Rule Enforcement
    System.out.println("\n>>> 4. TESTING SYSTEM ROBUSTNESS & CUSTOM EXCEPTION BOUNDARIES:");

    // Test Case A: Violating Savings Minimum Operating Balance Rule
    System.out.println("\n-- Test Case A: Savings Account Over-Withdrawal --");
    try {
        System.out.println("Attempting withdrawal of Rs. 45000.0 from Savings (Current
Balance: Rs. " + accSavings.getBalance() + ")...");
        accSavings.withdraw(45000.0);
    } catch (InsufficientFundsException ex) {
        System.out.println(">>> CAUGHT EXPECTED DOMAIN EXCEPTION: " + ex);
    } catch (Exception ex) {
        System.out.println(">>> Caught General Exception: " + ex);
    }

    // Test Case B: Violating Current Account Overdraft Ceiling Rule
    System.out.println("\n-- Test Case B: Current Account Overdraft Exhaustion --");
    try {
        System.out.println("Attempting additional withdrawal of Rs. 40000.0 from Current
Account...");
        accCorporate.withdraw(40000.0); // Exceeds remaining 30k overdraft limit
    } catch (OverdraftExceededException ex) {
        System.out.println(">>> CAUGHT EXPECTED DOMAIN EXCEPTION: " + ex);
    } catch (Exception ex) {
        System.out.println(">>> Caught General Exception: " + ex);
    }

    // 5. Applying Runtime Polymorphic Operations (Periodic Interest / Fee Accrual)
    System.out.println("\n>>> 5. EXECUTING RUNTIME POLYMORPHIC END-OF-QUARTER SERVICING:");
    Account[] portfolio = {accSavings, accCorporate, accThirdParty};

```

```

        for (Account acc : portfolio) {
            System.out.println("Processing Quarter-End for: " + acc.getAccountNumber() + " (" +
acc.getClass().getSimpleName() + ")");
            acc.applyPeriodicInterest(); // Dynamic Method Dispatch
            acc.generateAuditSummary();
        }

        // 6. Generating Official 2D Array Ledgers & Passbook Statements
        System.out.println("\n>>> 6. GENERATING PASSBOOK AUDIT STATEMENTS (2D ARRAY LEDGER):");
        accSavings.printStatement();
        accCorporate.printStatement();
        accThirdParty.printStatement();

        System.out.println("=====
        =");
        System.out.println("CAPSTONE PROJECT EXECUTION COMPLETED: ALL INTEGRATED OOP SUBSYSTEMS
        VERIFIED OK");

        System.out.println("=====
        =");
    }
}

```

■ CONCLUSION

Successfully designed and implemented the Smart Banking Transaction and Account Portfolio Management System (SB-TMS) as an integrated capstone project. Demonstrated seamless synergy of OOP encapsulation, inheritance hierarchies, abstract classes, multiple interfaces, compile-time and runtime polymorphism, multi-dimensional array ledgers, and robust custom exception handling in an enterprise banking scenario.

OUTPUT PRINT SECTION

Consolidated Laboratory Practical Outputs (Experiments 1 to 11) for Assessment and Archival

Experiment 1 – Output

DEMONSTRATE CONTROL STRUCTURE: IF-ELSE, SWITC...

Windows PowerShell - javac Exp1ControlStructures.java; java Exp1ControlStructures

```
PS C:\Users\anish\Downloads\java\record_project> javac Exp1ControlStructures.java; java
Exp1ControlStructures
```

```
=== RAIT Nerul - SBL OOP Java Lab: Experiment 1 ===
Candidate: Anish Vyapari | Roll No: 25CA1012
Topic: Control Structures and Class/Object Construction
```

```
=====
      STUDENT ACADEMIC PROFILE RECORD
=====
Roll Number : 1012
Name       : Anish Vyapari
Attendance : 91.5%

--- 1. Evaluating Eligibility (if-else structure) ---
Status: Eligible for Semester Examination (Attendance: 91.5% ≥ 75%)
```

```
--- 2. Calculating Academic Performance (for loop) ---
Subject 1 Marks: 92
Subject 2 Marks: 88
Subject 3 Marks: 95
Subject 4 Marks: 89
Subject 5 Marks: 94
Total Marks Obtained: 458 / 500
Calculated Average: 91.60%
Highest Subject Mark: 95

--- 3. Assigning Grade and Feedback (switch statement) ---
Letter Grade Assigned: 0
Faculty Remark: Outstanding Performance! Recommended for Merit Honors.
```

```
--- 4. Integrity Check / Digit Verification (while loop) ---
Student Roll No: 1012
Calculated Checksum of Roll Number: 4
Record Validation Status: Verified & Recorded in System.
```

```
-----
=====
      STUDENT ACADEMIC PROFILE RECORD
=====
Roll Number : 1045
Name       : Rohan Sharma
Attendance : 68.0%

--- 1. Evaluating Eligibility (if-else structure) ---
Status: Conditionally Eligible with Medical Condonation (Attendance: 68.0%)
```

```
--- 2. Calculating Academic Performance (for loop) ---
Subject 1 Marks: 54
Subject 2 Marks: 48
Subject 3 Marks: 62
Subject 4 Marks: 58
Subject 5 Marks: 50
Total Marks Obtained: 272 / 500
Calculated Average: 54.40%
Highest Subject Mark: 62
```

```
--- 3. Assigning Grade and Feedback (switch statement) ---  
Letter Grade Assigned: D  
Faculty Remark: Passed. Remedial sessions recommended in core modules.  
  
--- 4. Integrity Check / Digit Verification (while loop) ---  
Student Roll No: 1045  
Calculated Checksum of Roll Number: 10  
Record Validation Status: Verified & Recorded in System.  
=====
```

Windows PowerShell - javac Exp2Overloading.java; java Exp2Overloading

PS C:\Users\anish\Downloads\java\record_project> javac Exp2Overloading.java; java Exp2Overloading

=== RAIT Nerul - SBL OOP Java Lab: Experiment 2 ===

Candidate: Anish Vyapari | Roll No: 25CA1012

Topic: Method Overloading and Constructor Overloading

```
=====
PART A: DEMONSTRATION OF CONSTRUCTOR OVERLOADING
=====
[Constructor 1 Invoked]: Default zero-parameter constructor.
  Type: Undefined / Point Solid | Volume:      0.00 cu units
[Constructor 2 Invoked]: Single-parameter constructor (Side = 6.0).
  Type: Uniform Cube           | Volume:      216.00 cu units
[Constructor 3 Invoked]: Two-parameter constructor (Radius = 4.0, Height = 10.0).
  Type: Circular Cylinder      | Volume:      502.65 cu units
[Constructor 4 Invoked]: Three-parameter constructor (L = 8.0, B = 5.0, H = 3.0).
  Type: Rectangular Cuboid     | Volume:      120.00 cu units

=====
PART B: DEMONSTRATION OF METHOD OVERLOADING
=====
→ Executing: computeArea(double side) [Square]
  Calculated Square Area      : 56.25 sq units

→ Executing: computeArea(float radius) [Circle]
  Calculated Circle Area      : 78.54 sq units

→ Executing: computeArea(double length, double breadth) [Rectangle]
  Calculated Rectangle Area   : 78.00 sq units

→ Executing: computeArea(double base, double height, boolean) [Triangle]
  Calculated Triangle Area    : 40.00 sq units

→ Executing: computeArea(sideA, sideB, h, label) [Trapezoid]
  Calculated Trapezoid Area    : 72.00 sq units
=====
```

Windows PowerShell - javac Exp3Arrays.java; java Exp3Arrays

PS C:\Users\anish\Downloads\java\record_project> javac Exp3Arrays.java; java Exp3Arrays

=== RAIT Nerul - SBL OOP Java Lab: Experiment 3 ===

Candidate: Anish Vyapari | Roll No: 25CA1012

Topic: One-Dimensional (1D) and Two-Dimensional (2D) Arrays

```
=====
PART 1: ONE-DIMENSIONAL (1D) ARRAY OPERATIONS
=====
Original 1D Array Elements: [ 45, 88, 72, 95, 60, 34, 82, 91, 53, 76 ]
Array Length: 10
Sum of Elements      : 696
Mean / Average Value : 69.60
Minimum Element      : 34
Maximum Element      : 95
Linear Search: Key 82 found at Index 6
Sorted 1D Array (Ascending): [ 34, 45, 53, 60, 72, 76, 82, 88, 91, 95 ]
```

```
=====
PART 2: TWO-DIMENSIONAL (2D) ARRAY OPERATIONS
=====
```

Matrix A (3x3):

```
| 3  8  2 |
| 4  6  9 |
| 1  5  7 |
```

Matrix B (3x3):

```
| 5  2  7 |
| 8  1  3 |
| 6  4  2 |
```

Matrix Sum (Matrix A + Matrix B):

```
| 8 10  9 |
|12  7 12 |
| 7  9  9 |
```

Transpose of Matrix A (A^T):

```
| 3  4  1 |
| 8  6  5 |
| 2  9  7 |
```

Row and Column Diagnostics of Matrix A:

Row 0 Sum: 13

Row 1 Sum: 19

Row 2 Sum: 13

Principal Diagonal Sum (Trace): 16

```
=====
```

Windows PowerShell - javac Exp4Strings.java; java Exp4Strings

PS C:\Users\anish\Downloads\java\record_project> javac Exp4Strings.java; java Exp4Strings

=== RAIT Nerul - SBL OOP Java Lab: Experiment 4 ===

Candidate: Anish Vyapari | Roll No: 25CA1012

Topic: String Manipulation Functions and Palindrome Verification

=====

PART 1: STRING MANIPULATION FUNCTIONS

=====

Original String literal: " Object Oriented Programming in Java "

1. Length of string: 39
2. After trim(): "Object Oriented Programming in Java" (Length: 35)
3. Character at index 7: 'O'
4. Substring (0 to 15): "Object Oriented"
 Substring (from 16): "Programming in Java"
5. toUpperCase(): "OBJECT ORIENTED PROGRAMMING IN JAVA"
 toLowerCase(): "object oriented programming in java"
6. Index of 'Java': 31
 Index of character 'e': 3
7. replace("Java", "Modern Java 21"): "Object Oriented Programming in Modern Java 21"
8. concat(" - RAIT Lab"): "Object Oriented Programming in Java - RAIT Lab"
9. startsWith("Object"): true
 endsWith("Java"): true
10. Comparing "JavaProgramming" with "javaprogramming":
 equals() : false
 equalsIgnoreCase() : true

=====

PART 2: PALINDROME STRING VERIFICATION

=====

Input: "Radar"	"	Two-Pointer: true	StringBuilder: true	Result:
PALINDROME				
Input: "Madam"	"	Two-Pointer: true	StringBuilder: true	Result:
PALINDROME				
Input: "A man a plan a canal Panama"	"	Two-Pointer: true	StringBuilder: true	Result:
PALINDROME				
Input: "Was it a car or a cat I saw"	"	Two-Pointer: true	StringBuilder: true	Result:
PALINDROME				
Input: "Computer Science"	"	Two-Pointer: false	StringBuilder: false	Result: NOT A
PALINDROME				
Input: "12321"	"	Two-Pointer: true	StringBuilder: true	Result:
PALINDROME				
Input: "ObjectOriented"	"	Two-Pointer: false	StringBuilder: false	Result: NOT A
PALINDROME				

=====

Windows PowerShell - javac Exp5Inheritance.java; java Exp5Inheritance

PS C:\Users\anish\Downloads\java\record_project> javac Exp5Inheritance.java; java Exp5Inheritance

=== RAIT Nerul - SBL OOP Java Lab: Experiment 5 ===

Candidate: Anish Vyapari | Roll No: 25CA1012

Topic: Single and Multilevel Inheritance with 'super' Keyword

=====
PHASE 1: SINGLE INHERITANCE (Employee IS-A Person)
=====

Instantiating Employee object (Constructor Chaining):

[Person Constructor]: Initialized personal profile for Rajesh Kulkarni

[Employee Constructor]: Initialized employee credentials (ID: EMP-2041)

Invoking Employee Profile (demonstrating super.displayProfile()):

Personal Info → Name: Rajesh Kulkarni | Age: 28 | Blood Group: O+

Employment Info → Emp ID: EMP-2041 | Dept: Quality Assurance | Base Salary: Rs. 55000.0

Calculated Monthly Net Salary: Rs. 64900.00

=====
PHASE 2: MULTILEVEL INHERITANCE (TechLead IS-A Employee IS-A Person)
=====

Instantiating TechnicalProjectLead object (3-Tier Chaining):

[Person Constructor]: Initialized personal profile for Anish Vyapari

[Employee Constructor]: Initialized employee credentials (ID: LEAD-1012)

[TechnicalProjectLead Constructor]: Initialized leadership role for Autonomous Edge Core

Invoking Multilevel Display Hierarchy:

Personal Info → Name: Anish Vyapari | Age: 21 | Blood Group: B+

Employment Info → Emp ID: LEAD-1012 | Dept: AI & Systems Engineering | Base Salary: Rs. 95000.0

Leadership Info → Project: Autonomous Edge Core | Team Size: 8 Engineers | Project Allowance: Rs. 25000.0

Total Gross Remuneration: Rs. 137100.00

=====

```
Windows PowerShell - javac Exp6Interfaces.java; java Exp6Interfaces

PS C:\Users\anish\Downloads\java\record_project> javac Exp6Interfaces.java; java Exp6Interfaces

=== RAIT Nerul - SBL OOP Java Lab: Experiment 6 ===
Candidate: Anish Vyapari | Roll No: 25CA1012
Topic: Multiple Inheritance Concept using Java Interfaces

=====
CREATING INSTANCE OF CLASS IMPLEMENTING MULTIPLE INTERFACES
=====

--- Invoking Printable Interface Capabilities ---
[Print Service]: Page Layout configured to Portrait orientation.
[Print Service]: Printing Report → ID: RAIT-SBL-2026-001
Title       : "Advanced OOP Architecture and Design Patterns in Java"
Author      : Anish Vyapari
Total Pages : 18 pages on A4 (210 x 297 mm)
Status      : Output dispatched to high-resolution spooler successfully.

--- Invoking SecureCloudSharable Interface Capabilities ---
[Cloud Service]: Initializing encrypted payload transfer...
Destination   : https://cloud.rait.ac.in/vault/sbl-java/records
Cipher Mode   : AES-256 GCM
Transfer      : Secure handshake established. File transmitted successfully.
[Security]: Verifying SHA-256 integrity hash → Match Result: true

--- Invoking Auditable Interface Capabilities ---
[Audit Trail]: Access granted to user [Anish_Vyapari_25CA1012] at timestamp: 2026-09-28 10:15:30 IST

--- Demonstrating Polymorphic Interface References (Upcasting) ---
Printable Reference → Standard Format: A4 (210 x 297 mm)
[Print Service]: Printing Report → ID: RAIT-SBL-2026-001
Title       : "Advanced OOP Architecture and Design Patterns in Java"
Author      : Anish Vyapari
Total Pages : 18 pages on A4 (210 x 297 mm)
Status      : Output dispatched to high-resolution spooler successfully.
SecureCloudSharable Reference → Encryption: AES-256 GCM
[Cloud Service]: Initializing encrypted payload transfer...
Destination   : https://backup-node.rait.ac.in/replica
Cipher Mode   : AES-256 GCM
Transfer      : Secure handshake established. File transmitted successfully.
=====
```

Windows PowerShell - javac Exp7ExceptionHandling.java; java Exp7ExceptionHandling

```
PS C:\Users\anish\Downloads\java\record_project> javac Exp7ExceptionHandling.java; java Exp7ExceptionHandling
```

```
=== RAIT Nerul - SBL OOP Java Lab: Experiment 7 ===
```

```
Candidate: Anish Vyapari | Roll No: 25CA1012
```

```
Topic: Exception Handling using try, catch, throw, throws, and finally
```

```
Initial Account State: Holder: Anish Vyapari | Balance: Rs. 25000.0
```

```
=====
TRANSACTION CYCLE #1
=====
```

```
>>> Entering [try] block...
```

```
  [Attempting Withdrawal]: Request for Rs. 5000.0 on Account AC-1012-DYP
```

```
  Withdrawal Successful! Dispensed: Rs. 5000.00 | Remaining Balance: Rs. 20000.00
```

```
>>> Transaction processed without runtime exceptions.
```

```
>>> [finally] Block Executed: Audit log committed. DB connection flushed. Session state intact.
```

```
>>> End of Cycle Balance: Rs. 20000.00
```

```
=====
TRANSACTION CYCLE #2
=====
```

```
>>> Entering [try] block...
```

```
  [Attempting Withdrawal]: Request for Rs. -1200.0 on Account AC-1012-DYP
```

```
>>> [catch - IllegalArgumentException Caught]: Withdrawal amount must be strictly greater than zero! (Supplied: -1200.0)
```

```
>>> [finally] Block Executed: Audit log committed. DB connection flushed. Session state intact.
```

```
>>> End of Cycle Balance: Rs. 20000.00
```

```
=====
TRANSACTION CYCLE #3
=====
```

```
>>> Entering [try] block...
```

```
  [Attempting Withdrawal]: Request for Rs. 35000.0 on Account AC-1012-DYP
```

```
>>> [catch - ArithmeticException Caught]: Insufficient Balance! Requested: Rs. 35000.0, Available: Rs. 20000.0
```

```
>>> [finally] Block Executed: Audit log committed. DB connection flushed. Session state intact.
```

```
>>> End of Cycle Balance: Rs. 20000.00
```

```
=====
TRANSACTION CYCLE #4
=====
```

```
>>> Entering [try] block...
```

```
  [Attempting Withdrawal]: Request for Rs. 10000.0 on Account AC-1012-DYP
```

```
  Withdrawal Successful! Dispensed: Rs. 10000.00 | Remaining Balance: Rs. 10000.00
```

```
>>> Transaction processed without runtime exceptions.
```

```
>>> [finally] Block Executed: Audit log committed. DB connection flushed. Session state intact.
```

```
>>> End of Cycle Balance: Rs. 10000.00
```

```
=====
APPLICATION EXECUTION CONTINUES NORMALLY TO CONCLUSION
=====
```

Windows PowerShell - javac Exp8CustomExceptions.java; java Exp8CustomExceptions

PS C:\Users\anish\Downloads\java\record_project> javac Exp8CustomExceptions.java; java Exp8CustomExceptions

=== RAIT Nerul - SBL OOP Java Lab: Experiment 8 ===

Candidate: Anish Vyapari | Roll No: 25CA1012

Topic: Design and Implementation of User Defined Exceptions

=====

ENROLLMENT SUBMISSION REQUEST #1

=====

[Portal Processing Registration]: Candidate: Anish Vyapari, Age: 21, ID: MH-NERUL-101

Registration Approved!

[VOTER ID CARD ISSUED]

Card Number : MH-NERUL-101

Elector Name : Anish Vyapari

Age : 21 years

Constituency : Belapur / Nerul Division

Status : Active & Enrolled in Electoral Roll

=====

ENROLLMENT SUBMISSION REQUEST #2

=====

[Portal Processing Registration]: Candidate: Pooja Deshmukh, Age: 16, ID: MH-NERUL-102

[CAUGHT USER-DEFINED EXCEPTION]

Message : Eligibility Violation: Citizen must be at least 18 years old to exercise constitutional franchise.

Diagnostics: Given Age = 16, Required = 18

toString : InvalidVoterAgeException [ErrorCode: VOTE-ERR-001]: Eligibility Violation: Citizen must be at least 18 years old to exercise constitutional franchise. (Age Given: 16, Minimum Legal Age: 18)

=====

ENROLLMENT SUBMISSION REQUEST #3

=====

[Portal Processing Registration]: Candidate: Sameer Patil, Age: 24, ID: MH-NERUL-103

Registration Approved!

[VOTER ID CARD ISSUED]

Card Number : MH-NERUL-103

Elector Name : Sameer Patil

Age : 24 years

Constituency : Navi Mumbai Central

Status : Active & Enrolled in Electoral Roll

=====

ENROLLMENT SUBMISSION REQUEST #4

=====

[Portal Processing Registration]: Candidate: Anish Vyapari (Duplicate Re-entry), Age: 21, ID: MH-NERUL-101

[CAUGHT USER-DEFINED EXCEPTION]

Message : Integrity Violation: A voter record with this electoral identification number already exists in the system registry.

Diagnostics: Conflicting Voter ID = MH-NERUL-101

toString : DuplicateVoterRegistrationException [ErrorCode: VOTE-ERR-002]: Integrity Violation: A voter record with this electoral identification number already exists in the system registry. (Conflicting Voter ID: MH-NERUL-101)

=====

ALL ENROLLMENT CYCLES HANDLED SAFELY VIA CUSTOM EXCEPTIONS

=====

Figure 9.1: Authentic Java Swing GUI Window Execution Snapshot

RAMRAO ADIK INSTITUTE OF TECHNOLOGY, NERUL
Department of Computer Engineering | SBL - OOP Java GUI Workbench

Enrollment Input Controls (Components)

Full Name:

Roll Number:

PRN:

Gender: ☒ Male ☐ Female

Department:

Semester:

☒ SBL - OOP Java (Core)

Course Modul... ☒ Data Structures Lab

☐ Discrete Mathematical Structures

System Confirmation Ledger (Display Component)

```
=====
OFFICIAL ENROLLMENT CONFIRMATION
=====
Candidate Name   : Anish Vyapari
Roll Number     : 25CA1012
PRN Identifier   : DY25ENGU0AIM012
Gender          : Male
Branch / Stream  : Artificial Intelligence & Data
Academic Term   : Semester 3 (SE)
Enrolled Modules:
  [x] SBL - OOP Java (Core)
  [x] Data Structures Lab
Timestamp       : 2026-09-28 11:20:45 IST
Registration Status: SUCCESS - RECORD STORED IN
=====
```

GUI Status: Event Dispatch Thread Active | AWT/Swing Components Initialized

Windows PowerShell - javac Exp9SwingGUI.java; java Exp9SwingGUI

PS C:\Users\anish\Downloads\java\record_project> javac Exp9SwingGUI.java; java Exp9SwingGUI

RAIT Nerul - SBL OOP Java Lab: Experiment 9

Candidate: Anish Vyapari | Roll No: 25CA1012

Topic: Component and Container Classes for GUI Design in Java Swing

[GUI Event]: Form submitted and processed successfully for Anish Vyapari

GUI Snapshot successfully saved to: exp9_gui_window.png

Event Dispatch Thread (EDT) active. All Swing/AWT container and component classes rendered.

Windows PowerShell - javac Exp10DynamicScript.java; java Exp10DynamicScript

PS C:\Users\anish\Downloads\java\record_project> javac Exp10DynamicScript.java; java Exp10DynamicScript

=== RAIT Nerul - SBL OOP Java Lab: Experiment 10 ===

Candidate: Anish Vyapari | Roll No: 25CA1012

Topic: Dynamic Feature Implementation using Java-Based Scripting Engine

=====

DYNAMIC SCRIPT RULES LOADED AT RUNTIME:

=====

Rule 1 (Incentive Script):

"IF quarterlySales ≥ 500000 THEN basicSalary * 0.25 + 15000 ELSE basicSalary * 0.10 + 2000"

Rule 2 (Retention Bonus Script):

"IF clientRetentionRate ≥ 90 THEN 12000 ELSE 4000"

=====

EVALUATING DYNAMIC RULES ACROSS DOMAIN OBJECTS

=====

>>> Processing Executive: Anish Vyapari (ID: SE-101)

Profile: Base Salary = Rs. 75000.0 | Quarterly Sales = Rs. 850000.0 | Retention = 94% | Rating = 4.8

Executing Dynamic Incentive Calculation:

[Script Trace] Condition: (quarterlySales ≥ 500000) → Evaluated To: true

[Script Trace] Branch Taken: THEN → basicSalary * 0.25 + 15000

Executing Dynamic Retention Bonus Calculation:

[Script Trace] Condition: (clientRetentionRate ≥ 90) → Evaluated To: true

[Script Trace] Branch Taken: THEN → 12000

[Calculated Results via Dynamic Script]:

Base Salary : Rs. 75000.00

Dynamic Incentive: Rs. 33750.00

Retention Bonus : Rs. 12000.00

Total Payout : Rs. 120750.00

>>> Processing Executive: Priya Nair (ID: SE-102)

Profile: Base Salary = Rs. 60000.0 | Quarterly Sales = Rs. 420000.0 | Retention = 78% | Rating = 3.7

Executing Dynamic Incentive Calculation:

[Script Trace] Condition: (quarterlySales ≥ 500000) → Evaluated To: false

[Script Trace] Branch Taken: ELSE → basicSalary * 0.10 + 2000

Executing Dynamic Retention Bonus Calculation:

[Script Trace] Condition: (clientRetentionRate ≥ 90) → Evaluated To: false

[Script Trace] Branch Taken: ELSE → 4000

[Calculated Results via Dynamic Script]:

Base Salary : Rs. 60000.00

Dynamic Incentive: Rs. 8000.00

Retention Bonus : Rs. 4000.00

Total Payout : Rs. 72000.00

>>> Processing Executive: Vikram Rathod (ID: SE-103)

Profile: Base Salary = Rs. 65000.0 | Quarterly Sales = Rs. 620000.0 | Retention = 88% | Rating = 4.2

Executing Dynamic Incentive Calculation:

[Script Trace] Condition: (quarterlySales ≥ 500000) → Evaluated To: true

[Script Trace] Branch Taken: THEN → basicSalary * 0.25 + 15000

Executing Dynamic Retention Bonus Calculation:

[Script Trace] Condition: (clientRetentionRate ≥ 90) → Evaluated To: false

[Script Trace] Branch Taken: ELSE → 4000

[Calculated Results via Dynamic Script]:

Base Salary : Rs. 65000.00

Dynamic Incentive: Rs. 31250.00

Retention Bonus : Rs. 4000.00

Total Payout : Rs. 100250.00

```
=====
DYNAMIC SCRIPT EXECUTION COMPLETED SUCCESSFULLY
=====
```

Windows PowerShell - javac Exp11CapstoneProject.java; java Exp11CapstoneProject

PS C:\Users\anish\Downloads\java\record_project> javac Exp11CapstoneProject.java; java Exp11CapstoneProject

```
=====
  RAMRAO ADIK INSTITUTE OF TECHNOLOGY, NERUL - DEPARTMENT OF COMPUTER ENGG.
    SBL - OBJECT ORIENTED PROGRAMMING IN JAVA LABORATORY
    CAPSTONE PROJECT: SMART BANKING & TRANSACTION SYSTEM (SB-TMS)
  Candidate: Anish Vyapari | Roll No: 25CA1012 | PRN: DY25ENGU0AIM012
=====

>>> 1. INITIALIZING BANKING PORTFOLIO ACCOUNTS:
  → Accounts Provisioned Successfully in Core Banking Registry.

>>> 2. PERFORMING CORE TRANSACTIONS (DEPOSITS & WITHDRAWALS):
  [Deposit Confirmed]: Rs. 12500.00 deposited into [SB-1012-IN] (Semester Academic Scholarship
  Credit) | Balance: Rs. 62500.00
  [Deposit Confirmed]: Rs. 45000.00 deposited into [CA-9080-CORP] (Enterprise Client Invoice
  Settlement) | Balance: Rs. 145000.00
  [Savings Withdrawal]: Rs. 8000.00 dispensed from [SB-1012-IN] | Balance: Rs. 54500.00
  [Current Withdrawal]: Rs. 120000.00 debited from [CA-9080-CORP] | Balance: Rs. 25000.00

>>> 3. DEMONSTRATING INTER-ACCOUNT WIRE TRANSFER:

  [Initiating Inter-Account Wire Transfer]:
  From: SB-1012-IN (Anish Vyapari) → To: SB-3045-EXT (Kavita Rao) | Amount: Rs. 15000.00
  [Savings Withdrawal]: Rs. 15000.00 dispensed from [SB-1012-IN] | Balance: Rs. 39500.00
  [Deposit Confirmed]: Rs. 15000.00 deposited into [SB-3045-EXT] (Wire Inflow from SB-1012-IN) |
  Balance: Rs. 30000.00
  [Wire Transfer Completed Successfully!]

>>> 4. TESTING SYSTEM ROBUSTNESS & CUSTOM EXCEPTION BOUNDARIES:

-- Test Case A: Savings Account Over-Withdrawal --
Attempting withdrawal of Rs. 45000.0 from Savings (Current Balance: Rs. 39500.0)...
>>> CAUGHT EXPECTED DOMAIN EXCEPTION: InsufficientFundsException [ERR-BANK-001]: Savings Minimum
Balance Rule Violated (Required Threshold: Rs. 2000.0) (Available: Rs. 39500.0, Attempted: Rs.
45000.0)

-- Test Case B: Current Account Overdraft Exhaustion --
Attempting additional withdrawal of Rs. 40000.0 from Current Account...
[Current Overdraft Withdrawal]: Rs. 40000.00 debited. Current Overdraft Utilized: Rs. 15000.00 /
Rs. 50000.00

>>> 5. EXECUTING RUNTIME POLYMORPHIC END-OF-QUARTER SERVICING:
Processing Quarter-End for: SB-1012-IN (SavingsAccount)
  [Interest Credit]: Added quarterly interest Rs. 419.69 (@4.3% p.a.) | New Balance: Rs. 39919.69
  [Audit Metric]: Savings Account SB-1012-IN maintains compliance with minimum reserve.
Processing Quarter-End for: CA-9080-CORP (CurrentAccount)
  [Overdraft Levy]: Added overdraft utilization fee of Rs. 450.00 | Outstanding OD: Rs. 15450.00
  [Audit Metric]: Current Account CA-9080-CORP | Active Overdraft: Rs. 15450.00 / Rs. 50000.00
Processing Quarter-End for: SB-3045-EXT (SavingsAccount)
  [Interest Credit]: Added quarterly interest Rs. 300.00 (@4.0% p.a.) | New Balance: Rs. 30300.00
  [Audit Metric]: Savings Account SB-3045-EXT maintains compliance with minimum reserve.

>>> 6. GENERATING PASSBOOK AUDIT STATEMENTS (2D ARRAY LEDGER):

-----
OFFICIAL PASSBOOK STATEMENT | Account: SB-1012-IN | Holder: Anish Vyapari
-----
TXN ID      | TYPE      | AMOUNT (Rs.) | BALANCE (Rs.)
-----
TXN-0001    | OPEN      | 50000.00      | 50000.00
TXN-0002    | DEP       | 12500.00      | 62500.00
```

TXN-0003	WDL		8000.00		54500.00
TXN-0004	WDL		15000.00		39500.00
TXN-0005	INT		419.69		39919.69

Closing Net Balance: Rs. 39919.69

OFFICIAL PASSBOOK STATEMENT | Account: CA-9080-CORP | Holder: Vyapari Tech Solutions LLP

TXN ID	TYPE		AMOUNT (Rs.)		BALANCE (Rs.)
TXN-0001	OPEN		100000.00		100000.00
TXN-0002	DEP		45000.00		145000.00
TXN-0003	WDL		120000.00		25000.00
TXN-0004	WDL-OD		40000.00		0.00

Closing Net Balance: Rs. 0.00

OFFICIAL PASSBOOK STATEMENT | Account: SB-3045-EXT | Holder: Kavita Rao

TXN ID	TYPE		AMOUNT (Rs.)		BALANCE (Rs.)
TXN-0001	OPEN		15000.00		15000.00
TXN-0002	DEP		15000.00		30000.00
TXN-0003	INT		300.00		30300.00

Closing Net Balance: Rs. 30300.00

=====

CAPSTONE PROJECT EXECUTION COMPLETED: ALL INTEGRATED OOP SUBSYSTEMS VERIFIED OK

=====